



BERZEIT UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ENCS3320 - Computer Networks

Project #1

Socket Programming

Prepared by:

Hala Shojaieh #1231019

Mayar Etawi #1230501

Maryam Maslamani #1230199

Instructor: Dr. Ibrahim Nemer

Section: 4

Date: 24, May 2026

Table of Contents

Table of Contents	I
List of Tables	II
Table of Figures.....	III
Task 1 - CMD Networking Tools.....	1
Theoretical Background.....	1
Basic Understanding	2
Procedure	3
Results.....	10
Discussion and Analysis	10
Task 2 – TCP and UDP Client/Server Applications	11
Theoretical Background.....	11
TCP	12
UDP.....	16
Discussion and Analysis	20
Task 3 – Simple File-Sharing Web Server using Socket Programming	21
Theoretical Background.....	21
Procedure	22
Results, Discussion and Analysis	33
Conclusion	42
References.....	43

List of Tables

<i>Table 1:CMD commands and their purpose</i>	<i>2</i>
---	----------

Table of Figures

Figure 1: ipconfig /all output	3
Figure 2: ping output showing successful connectivity to the default gateway.	4
Figure 3: arp -a output showing ARP table entries on the local network	5
Figure 4: ping google.com successful internet connectivity using a domain name.....	5
Figure 5: ping 8.8.8.8 successful internet connectivity using a direct IP	6
Figure 6: tracert google.com output showing packet route and hop delays.....	7
Figure 7: nslookup google.com output - DNS resolution for Google's domain.....	7
Figure 8: nslookup facebook.com output - DNS resolution for Facebook's domain. ...	8
Figure 9: netstat -an output - active network connections and listening ports.....	9
Figure 10: TCP server program	12
Figure 11: TCP client program	13
Figure 12: Output of the TCP Server program	14
Figure 13: Output of the TCP Client program	14
Figure 14: Output of the TCP Client program - Client before Server	15
Figure 15: UDP server program.....	16
Figure 16: UDP client program.....	17
Figure 17: Output of the UDP server program – Client after Server.....	18
Figure 18: Output of UDP client program – Client after Server.....	18
Figure 19: Output of UDP server program – Client before Server.....	19
Figure 20: Output of UDP client program – Client before Server.....	19
Figure 21: Web Server program – Imports and Global Variables	22
Figure 22: Web Server program – get_content_Type() function.....	23
Figure 23: Web Server program – response() function	24
Figure 24: Web Server program – server_file() function	25
Figure 25: Web Server program – send_error() function	26
Figure 26: Web Server program – log() function	27
Figure 27: Web Server program – setup_server() function	28
Figure 28: Web Server program – Main Server Loop	29
Figure 29: Arabic Homepage of the File-Sharing Web Server (1).....	31
Figure 30: Arabic Homepage of the File-Sharing Web Server (2).....	32
Figure 31: English Homepage of the File-Sharing Web Server (1).....	32
Figure 32: English Homepage of the File-Sharing Web Server (2).....	32

Figure 33: Server startup.....	33
Figure 34: http://localhost:6501/en test	33
Figure 35: http://localhost:6501/home_en.html test	34
Figure 36: http://localhost:6501/index.html test	34
Figure 37: Terminal Output (1) – English test.....	34
Figure 38: Terminal Output (2) – English test.....	35
Figure 39: Terminal Output (3) – English test.....	35
Figure 40: Terminal Output (4) – English test.....	35
Figure 41: http://localhost:6501/ar test	36
Figure 42: http://localhost:6501/home_ar.html test.....	36
Figure 43: Terminal output (1) – Arabic test	37
Figure 44: Terminal Output (2) – Arabic test	37
Figure 45: Terminal Output (3) – Arabic test	37
Figure 46: 200 OK – Successful File Request in Browser	38
Figure 47: 200 OK – terminal output.....	38
Figure 48: 404 Not Found – Browser Response for Non-Existing File Request.....	39
Figure 49: 404 Not Found – Terminal Output	39
Figure 50: 400 Bad Request – Browser Response for Malformed Request	40
Figure 51: 404 Not Found – Terminal Output (1)	40
Figure 52: 404 Not Found – Terminal Output (2)	41
Figure 53: 404 Not Found – Terminal Output (3)	41

Task 1 - CMD Networking Tools

Theoretical Background

In this task, we use Command Prompt (CMD) networking tools to understand how computers communicate and how networks work. The ipconfig command shows the configuration of our network adapters, including IP address, subnet mask, default gateway, and DNS server. The ping command uses ICMP to test connectivity between devices by sending echo requests and measuring response time and packet loss. The tracert command traces the route packets take to reach a destination, showing each hop and its delay, which is useful for diagnosing routing issues. ICMP (Internet Control Message Protocol) is the underlying protocol used by ping and tracert to report errors and network problems. RTT (Round Trip Time) measures the time it takes for a packet to travel to a destination and back, indicating network speed and latency. The nslookup command allows querying DNS servers to find the IP addresses associated with domain names, while the DNS system translates human-readable names to IP addresses. The arp -a command shows the mapping of IP addresses to MAC addresses of devices on the local network. The netstat -an command lists all active connections and listening ports, helping monitor network activity. Finally, the telnet command allows connecting to remote devices on specific ports to test connectivity or services. These tools provide practical insight into network communication, help troubleshoot connectivity issues, understand packet flow, and lay the foundation for TCP/UDP protocols and HTTP request/response concepts. [1]

Basic Understanding

Table 1: CMD commands and their purpose

Command	Purpose
ipconfig	Displays the computer's network configuration, including IP address, subnet mask, default gateway, and DNS server.
ping	Tests connectivity to another device by sending echo requests and measuring response time and packet loss.
tracert	Traces the route packets take to reach a destination, showing each hop and its delay.
nslookup	Queries DNS servers to find the IP address for a domain name.
arp	Displays the ARP table, which maps IP addresses to MAC addresses on the local network.
netstat	Displays all active connections and listening ports on the computer.
telnet	Connects to a remote host on a specific port to test connectivity or service.

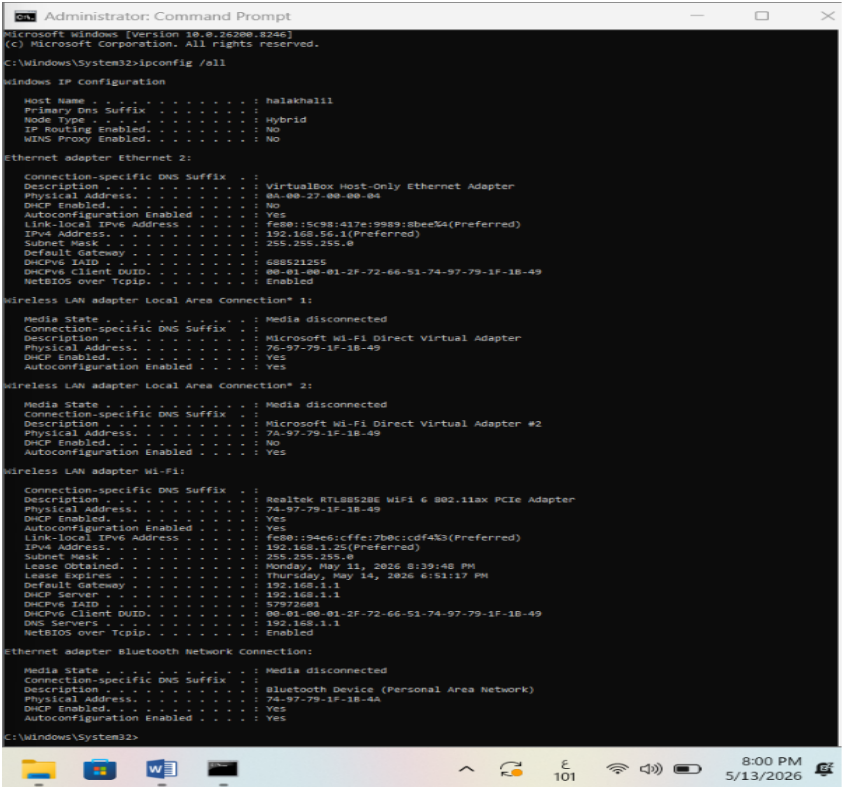
Procedure

The following section presents the practical procedure of Task 1, where we systematically utilize Command Prompt (CMD) networking tools to investigate the communication mechanisms of our computer within local and global networks. The commands executed include ipconfig, ping, tracert, nslookup, arp, and netstat. For each command, screenshots of the output are captured to document the results, accompanied by a brief explanation highlighting the purpose and significance of the observed data. This hands-on procedure provides an empirical understanding of network configuration, connectivity, routing, and diagnostic techniques, thereby reinforcing theoretical concepts with practical experience.

Local Network Exploration

Step 1: ipconfig /all

identify your computer's IPv4 address, default gateway, and DNS server.



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26280.2246]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\System32>ipconfig /all

Windows IP Configuration

Host Name . . . . . : halakhal11
Primary Dns Suffix . . . . . : 
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . : 
   Description . . . . . : VirtualBox Host-Only Ethernet Adapter
   Physical Address. . . . . : 08-00-27-00-00-00
   DHCP Enabled. . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::5c98:437e:9989:8bee%4(Preferred)
   IPv4 Address. . . . . : 192.168.56.1(Preferred)
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . . . : 
   DHCPv6 IAID . . . . . : 688521255
   DHCPv6 Client DUID. . . . . : 00-01-00-01-2F-72-66-51-74-97-79-1F-1B-49
   NetBIOS over Tcpip. . . . . : Enabled

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : 
   Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
   Physical Address. . . . . : 76-97-79-1F-1B-49
   DHCP Enabled. . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : 
   Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
   Physical Address. . . . . : 7A-97-79-1F-1B-49
   DHCP Enabled. . . . . : No
   Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : 
   Description . . . . . : Realtek RTL8852BE Wi-Fi 6 802.11ax PCIe Adapter
   Physical Address. . . . . : 74-97-79-1F-1B-49
   DHCP Enabled. . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::94e6:cffe:7b0c:cd43(Preferred)
   IPv4 Address. . . . . : 192.168.1.25(Preferred)
   Subnet Mask . . . . . : 255.255.255.0
   Lease Obtained. . . . . : Monday, May 11, 2026 8:39:48 PM
   Lease Expires . . . . . : Thursday, May 14, 2026 8:51:17 PM
   Default Gateway . . . . . : 192.168.1.1
   DHCP Server . . . . . : 192.168.1.1
   DHCPv6 IAID . . . . . : 57972683
   DHCPv6 Client DUID. . . . . : 00-01-00-01-2F-72-66-51-74-97-79-1F-1B-49
   DNS Servers . . . . . : 192.168.1.1
   NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : 
   Description . . . . . : Bluetooth Device (Personal Area Network)
   Physical Address. . . . . : 74-97-79-1F-1B-4A
   DHCP Enabled. . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

C:\Windows\System32>
```

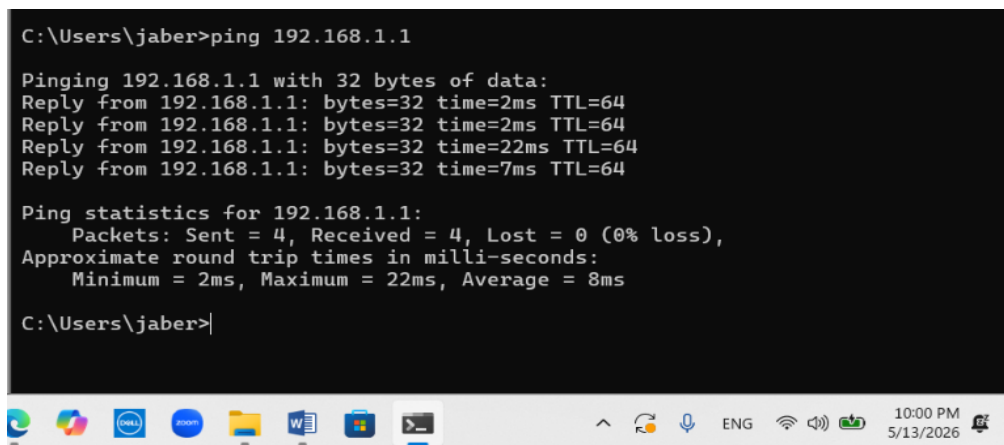
Figure 1: ipconfig /all output

Result Interpretation

The ipconfig /all output shows the computer's full network configuration. The active Wi-Fi adapter has IPv4 address 192.168.1.25, default gateway 192.168.1.1, and DNS server 192.168.1.1. This means the computer is correctly connected to the local network and can use the gateway and DNS server to access the internet.

Step 2: ping <default gateway>

Test connectivity with the local gateway and observe response time and packet loss.



```
C:\Users\jaber>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64
Reply from 192.168.1.1: bytes=32 time=22ms TTL=64
Reply from 192.168.1.1: bytes=32 time=7ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 22ms, Average = 8ms

C:\Users\jaber>
```

Figure 2: ping output showing successful connectivity to the default gateway.

Result Interpretation

The ping 192.168.1.1 command was used to test the connection with the default gateway. The result shows that 4 packets were sent and 4 were received with 0% loss, which means the local connection is working properly. The response times were low, with an average of 8 ms, indicating a stable connection within the local network.

Step 3: arp -a

Display ARP table entries for devices recently communicating with the computer on the local network.

```
Administrator: Command Prompt
C:\Windows\System32>arp -a

Interface: 192.168.1.25 --- 0x3
Internet Address      Physical Address      Type
192.168.1.1           68-58-11-ed-dd-50    dynamic
192.168.1.50          7c-27-bc-3a-b4-d4    dynamic
192.168.1.255         ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.102.18        01-00-5e-7f-66-12    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.1 --- 0x4
Internet Address      Physical Address      Type
192.168.56.255        ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static
```

Figure 3: arp -a output showing ARP table entries on the local network

Result Interpretation

The arp -a command displays the ARP table, which maps IP addresses to MAC addresses on the local network. The output shows the active interface 192.168.1.25 and includes the default gateway 192.168.1.1 with its MAC address. The dynamic entries represent devices recently communicating with the computer, while most static entries shown are broadcast or multicast addresses.

Internet Connectivity Tests

Step 1: ping google.com

Test connectivity to a remote server using its domain name.

```
C:\Users\jaber>ping google.com

Pinging google.com [142.250.75.206] with 32 bytes of data:
Reply from 142.250.75.206: bytes=32 time=8ms TTL=119
Reply from 142.250.75.206: bytes=32 time=41ms TTL=119
Reply from 142.250.75.206: bytes=32 time=15ms TTL=119
Reply from 142.250.75.206: bytes=32 time=105ms TTL=119

Ping statistics for 142.250.75.206:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 8ms, Maximum = 105ms, Average = 42ms

C:\Users\jaber>
```

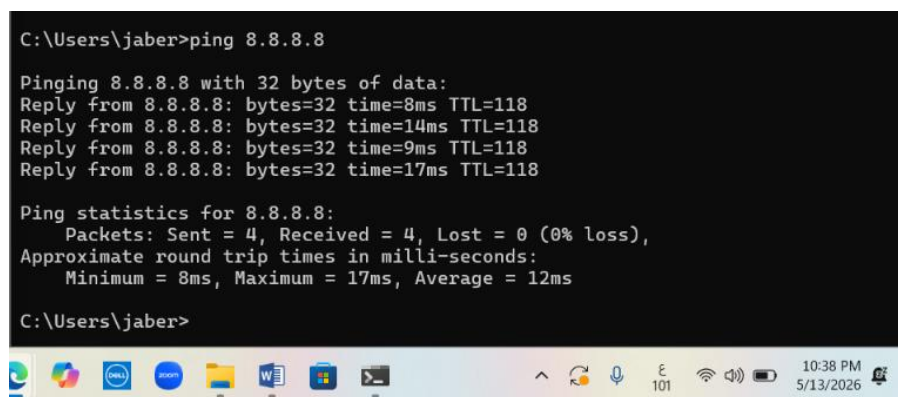
Figure 4: ping google.com successful internet connectivity using a domain name

Result Interpretation

The ping google.com command was used to test internet connectivity using a domain name. The result shows that the domain was resolved successfully and the packets were received with 0% loss. This confirms that the computer has internet access and that DNS is working correctly.

Step 2: ping 8.8.8.8

Test internet connectivity using a direct IP address.

A screenshot of a Windows command prompt window. The title bar shows 'C:\Users\jaber>'. The command 'ping 8.8.8.8' has been entered. The output shows four successful replies from 8.8.8.8 with varying response times (8ms, 14ms, 9ms, 17ms) and a TTL of 118. Below the replies, the ping statistics are displayed: 'Ping statistics for 8.8.8.8: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss), Approximate round trip times in milli-seconds: Minimum = 8ms, Maximum = 17ms, Average = 12ms'. The prompt 'C:\Users\jaber>' is visible at the bottom of the command window. The Windows taskbar is visible at the bottom of the screen, showing various application icons and the system clock indicating 10:38 PM on 5/13/2026.

```
C:\Users\jaber>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=8ms TTL=118
Reply from 8.8.8.8: bytes=32 time=14ms TTL=118
Reply from 8.8.8.8: bytes=32 time=9ms TTL=118
Reply from 8.8.8.8: bytes=32 time=17ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 8ms, Maximum = 17ms, Average = 12ms

C:\Users\jaber>
```

Figure 5: ping 8.8.8.8 successful internet connectivity using a direct IP

Result Interpretation

The ping 8.8.8.8 command was used to test internet connectivity using a direct IP address. The result shows that 4 packets were sent and 4 were received with 0% loss, which confirms successful communication with Google's DNS server. Since this test uses an IP address directly, it does not require DNS resolution, and the low average response time of 12 ms indicates a stable internet connection.

Comparison between Domain Name and IP Address

The ping google.com command tested internet connectivity using a domain name, so DNS resolution was needed before communication. In contrast, ping 8.8.8.8 used a direct IP address and did not require DNS. Both tests returned replies with 0% packet loss, confirming stable internet connectivity, while the domain-based test also confirmed that DNS resolution is working correctly.

Step 3: tracert google.com

Trace the route packets take to reach google.com and observe the number of hops and delays.

```
C:\Users\jaber>tracert google.com

Tracing route to google.com [142.250.75.206]
over a maximum of 30 hops:

  1  122 ms    14 ms    12 ms    192.168.1.1
  2   12 ms     7 ms    43 ms    10.74.32.250
  3   72 ms    27 ms    65 ms    10.74.19.201
  4   44 ms    61 ms   113 ms   142.251.206.188
  5  743 ms   743 ms   175 ms   142.250.234.183
  6   28 ms    51 ms    34 ms   142.251.240.177
  7   20 ms    19 ms    21 ms   lctlvq-ac-in-f14.1e100.net [142.250.75.206]

Trace complete.
C:\Users\jaber>
```

Figure 6: tracert google.com output showing packet route and hop delays

Result Interpretation

The tracert google.com command was used to trace the path from the computer to Google's server. The output shows that the packets reached google.com successfully in 7 hops, passing through several routers before reaching the destination. The response times are different for each hop, which indicates normal network delay along the route.

DNS Investigation

Step 1: nslookup google.com

Investigate DNS resolution for the domain google.com.

```
C:\Users\jaber>nslookup google.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4028:80a::200e
           142.250.75.206

C:\Users\jaber>
```

Figure 7: nslookup google.com output - DNS resolution for Google's domain

Result Interpretation

The nslookup google.com command was used to check DNS resolution for the domain name google.com. The output shows that the DNS server 192.168.1.1 successfully returned Google's IPv4 address 142.250.75.206 and an IPv6 address. This confirms that DNS resolution is working correctly and the computer can translate domain names into IP addresses.

Step 2: nslookup facebook.com

Investigate DNS resolution for the domain facebook.com.

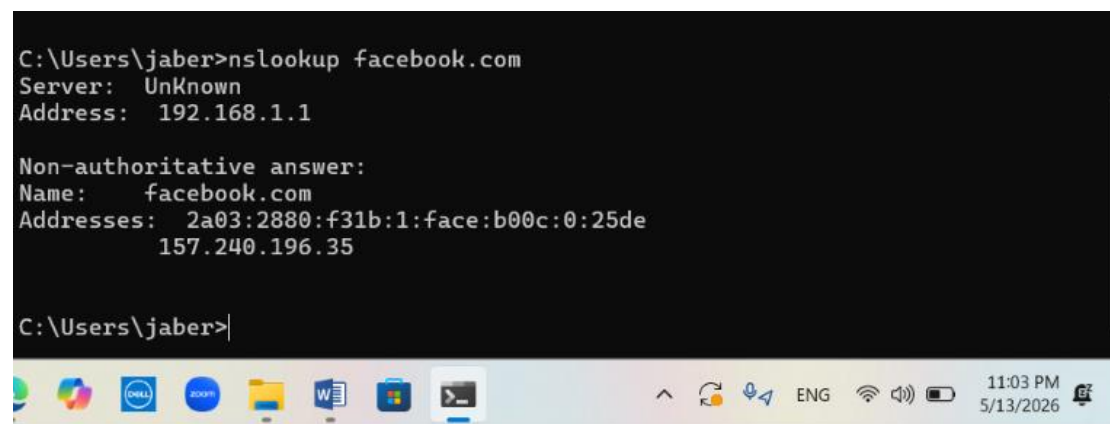
A screenshot of a Windows command prompt window. The title bar is not visible. The command prompt shows the user 'jaber' at the 'C:\Users\' location. The command 'nslookup facebook.com' has been entered. The output shows the DNS server as 'UnKnown' with address '192.168.1.1'. It then displays a 'Non-authoritative answer:' for 'facebook.com', listing two IP addresses: an IPv6 address '2a03:2880:f31b:1:face:b00c:0:25de' and an IPv4 address '157.240.196.35'. The prompt is currently at 'C:\Users\jaber>'. The taskbar at the bottom shows various icons including a globe, a folder, a Word document, a folder, a taskbar icon, and system tray icons for network, volume, and battery. The system clock shows '11:03 PM 5/13/2026'.

Figure 8: nslookup facebook.com output - DNS resolution for Facebook's domain.

Result Interpretation

The nslookup facebook.com command was used to check DNS resolution for the domain name facebook.com. The output shows that the DNS server 192.168.1.1 successfully returned Facebook's IPv4 address 157.240.196.35 and an IPv6 address. This confirms that DNS resolution is working correctly and that different domain names are mapped to different IP addresses.

Results

The executed CMD networking commands successfully demonstrated local network configuration, internet connectivity, DNS resolution, routing paths, ARP table mappings, and active network connections. The ping tests showed stable communication with 0% packet loss and low response times, confirming reliable local and internet connectivity. Other commands provided useful information about the network path, DNS servers, connected devices, and current network activity.

Discussion and Analysis

This task provided practical experience with CMD networking tools and showed how computers communicate within local and external networks. Commands such as ipconfig, ping, tracert, nslookup, arp, and netstat were used to examine IP configuration, connectivity, DNS resolution, packet routing, ARP mappings, and active connections.

The experiments confirmed that the computer was correctly configured and able to communicate with both the local gateway and external servers. The comparison between ping google.com and ping 8.8.8.8 highlighted the role of DNS in translating domain names into IP addresses. In the tracert result, one hop showed a high delay of 743 ms, which may be caused by temporary congestion or routing delay. Overall, these tools are useful for troubleshooting network problems and monitoring network activity.

Task 2 – TCP and UDP Client/Server Applications

Theoretical Background

Network applications commonly use the client-server communication model, where the client initiates communication and the server waits for requests from clients. Communication between applications is achieved through socket programming, where a socket acts as an interface between the application layer and the transport layer. Each communicating process is identified using an IP address and a port number, allowing data exchange between different hosts over a network. Socket programming enables applications to send and receive messages through transport layer protocols such as TCP and UDP.

The Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are two common transport layer protocols used in network communication. TCP is a connection-oriented protocol that establishes a reliable connection between the client and the server before transmitting data, ensuring ordered and error-checked delivery. In contrast, UDP is a connectionless protocol that transmits independent packets called datagrams without establishing a connection, providing faster communication with lower overhead. Applications such as web browsing and file transfer commonly use TCP, while UDP is often used in online games, VoIP, DNS, and video streaming applications. [2]

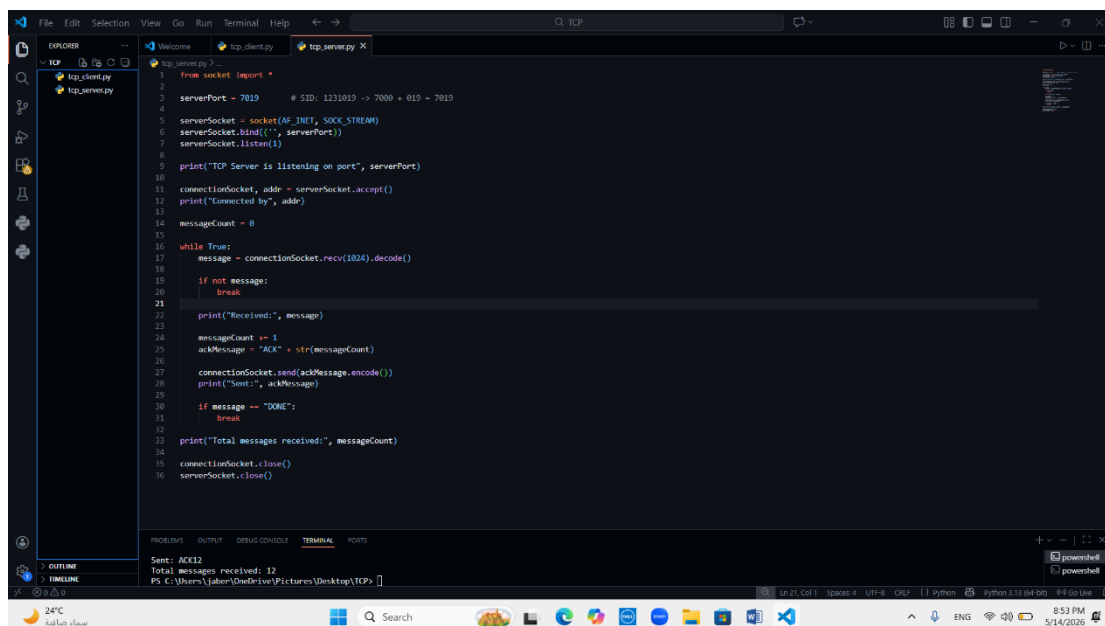
TCP

Procedure

In this part, TCP client and server applications were implemented using Python socket programming to simulate communication between two network processes. The server was configured to listen on port 7019, which was obtained by adding the last three digits of the student ID (1231019) to 7000. Both programs were developed using the Python socket library and the SOCK_STREAM socket type, which is used for TCP communication.

First, the TCP server program was implemented as shown in Figure (10) below. A TCP socket was created, bound to the specified port using the bind() function, and then placed in listening mode using listen(). The server accepted the client connection using accept() and continuously received messages using recv(). Each received message was displayed on the terminal and counted using a message counter.

After receiving each message, the server generated and sent an acknowledgment message such as ACK1, ACK2, and so on using the send() function. The communication continued until the message "DONE" was received. Finally, the server displayed the total number of received messages and closed the connection.



```
1 #true socket import *
2
3 serverPort = 7019
4 # SID: 1231019 -> 7000 + 019 = 7019
5
6 serverSocket = socket(AF_INET, SOCK_STREAM)
7 serverSocket.bind(('', serverPort))
8 serverSocket.listen()
9
10 print("TCP Server is listening on port", serverPort)
11
12 connectionSocket, addr = serverSocket.accept()
13 print("Connected by", addr)
14
15 messageCount = 0
16
17 while True:
18     message = connectionSocket.recv(1024).decode()
19
20     if not message:
21         break
22
23     print("Received:", message)
24
25     messageCount += 1
26     ackMessage = "ACK" + str(messageCount)
27
28     connectionSocket.send(ackMessage.encode())
29     print("Sent:", ackMessage)
30
31     if message == "DONE":
32         break
33
34 print("Total messages received:", messageCount)
35
36 connectionSocket.close()
37 serverSocket.close()
```

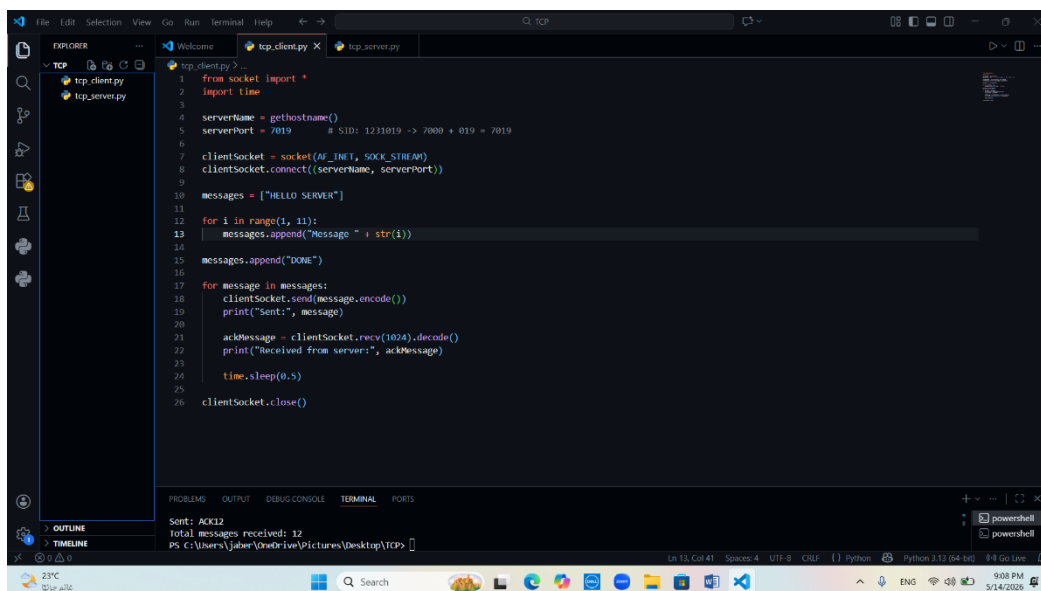
Terminal Output:

```
Sent: ACK12
Total messages received: 12
PS C:\Users\jaban\OneDrive\Pictures\Desktop\TCP>
```

Figure 10: TCP server program

Next, the TCP client program was implemented as shown in Figure (11) below. A TCP socket was created and connected to the server using the connect() function. The client generated and transmitted a sequence of messages starting with "HELLO SERVER", followed by 10 numbered messages in the format Message 1 to Message 10, and finally the "DONE" message to terminate the communication.

Each message was encoded and sent to the server using the send() function. After sending each message, the client waited to receive an acknowledgment message from the server using the recv() function. The received acknowledgment messages were then decoded and displayed on the terminal. In addition, a small delay using time.sleep(0.5) was added between message transmissions to make the communication process easier to observe during execution and testing.



```
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7019 # SID: 1231019 -> 7000 + 019 = 7019
6
7 clientSocket = socket(AF_INET, SOCK_STREAM)
8 clientSocket.connect((serverName, serverPort))
9
10 messages = ["HELLO SERVER"]
11
12 for i in range(1, 11):
13     messages.append("Message " + str(i))
14
15 messages.append("DONE")
16
17 for message in messages:
18     clientSocket.send(message.encode())
19     print("Sent:", message)
20
21     ackMessage = clientSocket.recv(1024).decode()
22     print("Received from server:", ackMessage)
23     time.sleep(0.5)
24
25 clientSocket.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

SENT: ACK12
Total messages received: 12
PS C:\Users\jaban\OneDrive\Pictures\Desktop\TCP>

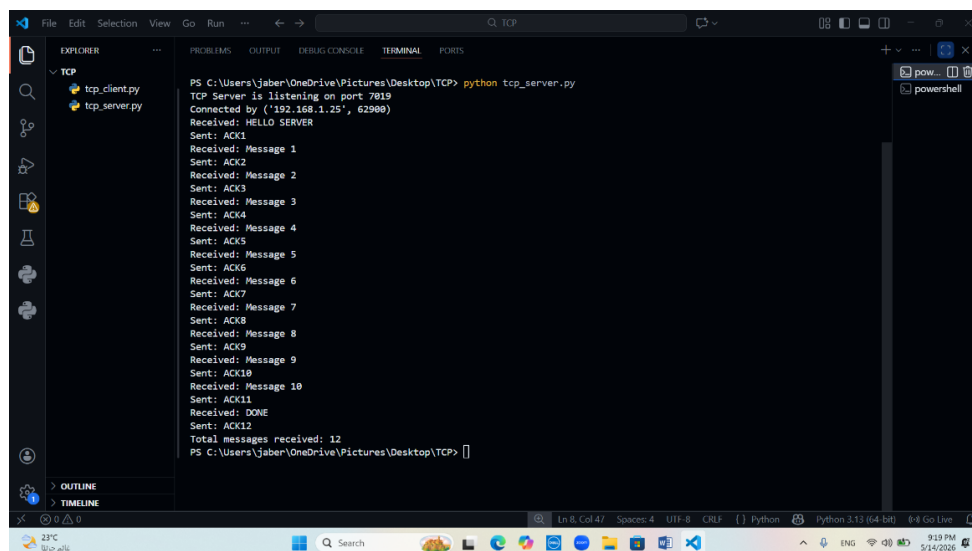
Figure 11: TCP client program

Results

Server before Client

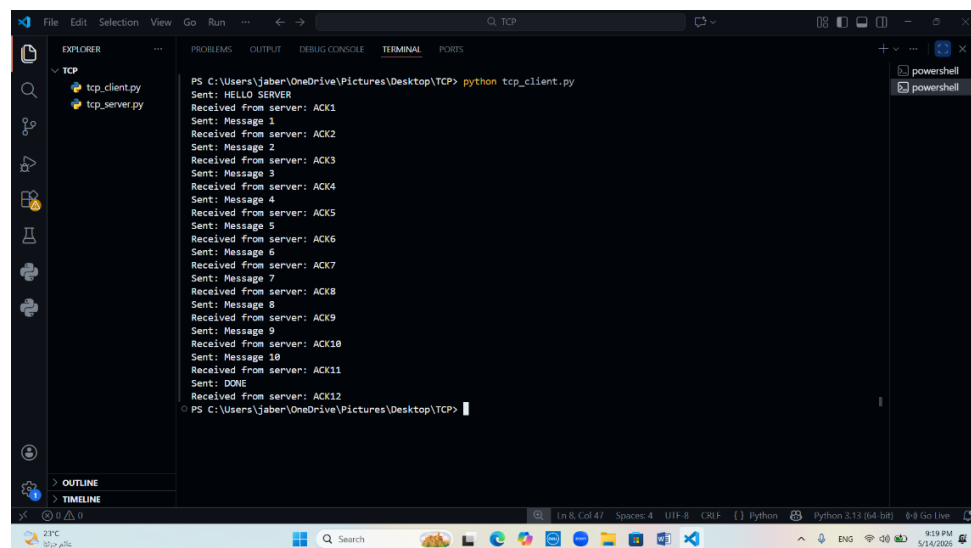
In the TCP test case, the server program was started before the client program. As shown in Figures (12) and (13), the client successfully connected to the server and transmitted all messages correctly, including “HELLO SERVER”, the 10 numbered messages, and the final “DONE” message. The server received all messages in the correct order without any missing data and sent acknowledgment messages

(ACK1–ACK12) after each received message. This demonstrates the reliable and ordered communication provided by the TCP protocol, ensuring successful delivery of all transmitted messages.



```
PS C:\Users\jaber\OneDrive\Pictures\Desktop\TCP> python tcp_server.py
TCP Server is listening on port 7019
Connected by ('192.168.1.25', 62900)
Received: HELLO SERVER
Sent: ACK1
Received: Message 1
Sent: ACK2
Received: Message 2
Sent: ACK3
Received: Message 3
Sent: ACK4
Received: Message 4
Sent: ACK5
Received: Message 5
Sent: ACK6
Received: Message 6
Sent: ACK7
Received: Message 7
Sent: ACK8
Received: Message 8
Sent: ACK9
Received: Message 9
Sent: ACK10
Received: Message 10
Sent: ACK11
Received: DONE
Sent: ACK12
Total messages received: 12
PS C:\Users\jaber\OneDrive\Pictures\Desktop\TCP> 
```

Figure 12:Output of the TCP Server program

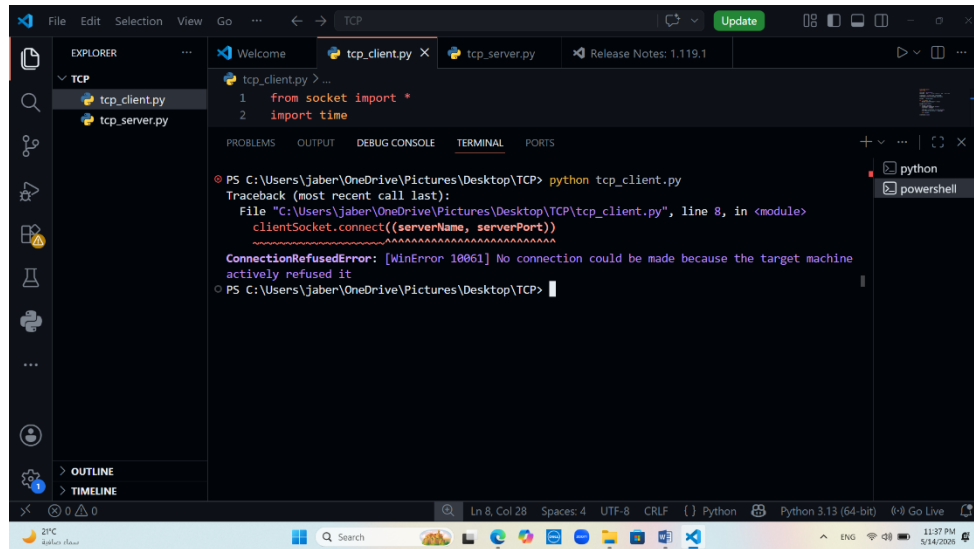


```
PS C:\Users\jaber\OneDrive\Pictures\Desktop\TCP> python tcp_client.py
Sent: HELLO SERVER
Received from server: ACK1
Sent: Message 1
Received from server: ACK2
Sent: Message 2
Received from server: ACK3
Sent: Message 3
Received from server: ACK4
Sent: Message 4
Received from server: ACK5
Sent: Message 5
Received from server: ACK6
Sent: Message 6
Received from server: ACK7
Sent: Message 7
Received from server: ACK8
Sent: Message 8
Received from server: ACK9
Sent: Message 9
Received from server: ACK10
Sent: Message 10
Received from server: ACK11
Sent: DONE
Received from server: ACK12
PS C:\Users\jaber\OneDrive\Pictures\Desktop\TCP> 
```

Figure 13:Output of the TCP Client program

Client before Server

In the TCP client-before-server test case, the client program was started before the server. As shown in Figure (14), the client failed to establish a connection and displayed a `ConnectionRefusedError`. This occurred because TCP is connection-oriented and requires the server to be running and listening before the client can connect. Therefore, no messages were transmitted in this case.



The screenshot shows the Visual Studio Code interface with a file explorer on the left containing 'tcp_client.py' and 'tcp_server.py'. The main editor displays the code for 'tcp_client.py':

```
1 from socket import *
2 import time
```

The terminal at the bottom shows the command `python tcp_client.py` being executed. The output is a traceback showing a `ConnectionRefusedError: [WinError 10061] No connection could be made because the target machine actively refused it`. The error message is repeated multiple times, indicating a loop in the code. The status bar at the bottom indicates the file is at line 8, column 28, and the Python interpreter is set to Python 3.13 (64-bit).

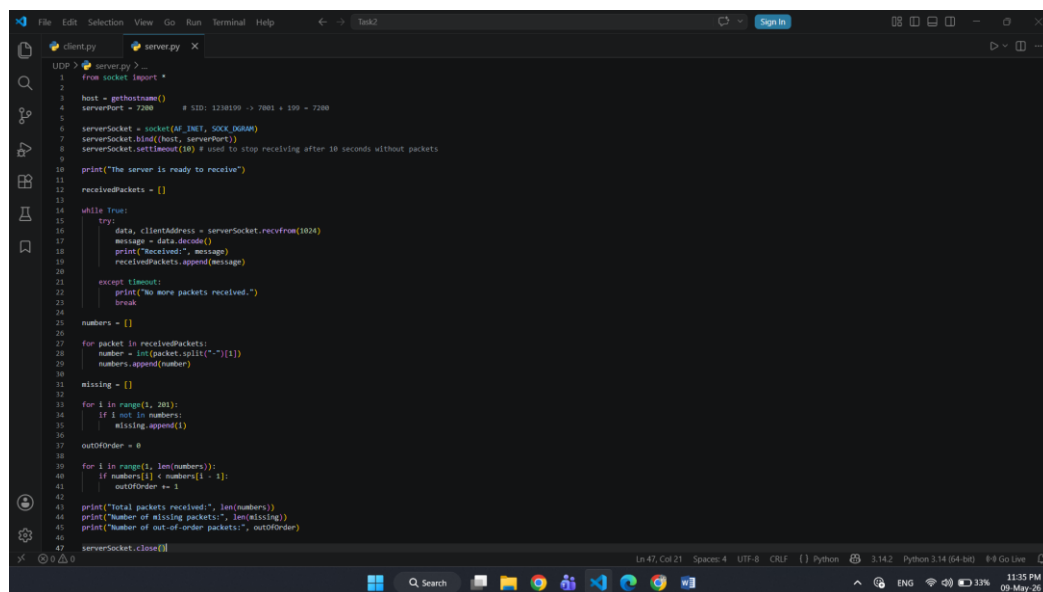
Figure 14: Output of the TCP Client program - Client before Server

UDP

Procedure

In this part, UDP client and server applications were implemented using Python socket programming to simulate communication between two network processes. The server was configured to listen on port 7200, which was obtained by adding the last three digits of the student ID (1230199) to 7001 according to the project requirements. Both programs were developed using the Python socket library and the SOCK_DGRAM socket type, which is used for UDP communication.

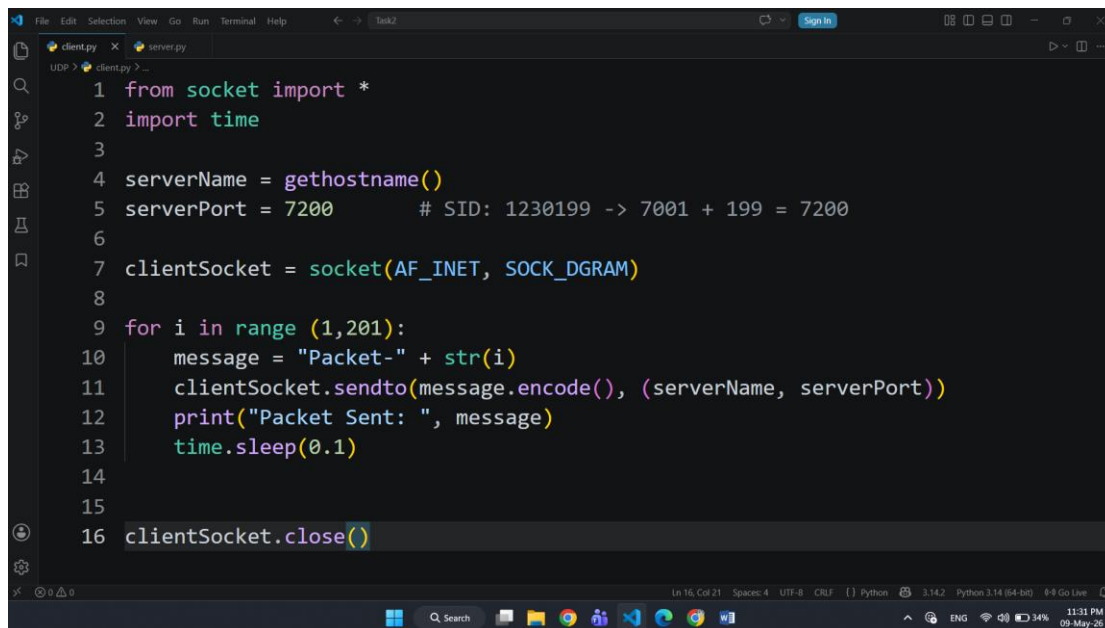
First, the UDP server program was implemented as shown in Figure (15) below. A UDP socket was created and then bound to the local host and port number using the bind() function. The server continuously waited for incoming packets from the client using the recvfrom() function. Each received packet was decoded, displayed on the terminal, and stored in a list for later analysis. After packet reception was completed, the server extracted the sequence number from each received packet and compared the received packet numbers with the expected sequence from 1 to 200 in order to determine the total number of received packets and detect any missing packets. In addition, the received packet order was analyzed by comparing consecutive sequence numbers to identify out-of-order packets. A timeout mechanism using settimeout() was implemented to stop the server after a period of inactivity.



```
1 from socket import *
2
3 host = gethostname()
4 serverPort = 7200 # SID: 1230199 -> 7001 + 199 = 7200
5
6 serverSocket = socket(AF_INET, SOCK_DGRAM)
7 serverSocket.bind((host, serverPort))
8 serverSocket.settimeout(10) # used to stop receiving after 10 seconds without packets
9
10 print("The server is ready to receive")
11
12 receivedPackets = []
13
14 while True:
15     try:
16         data, clientAddress = serverSocket.recvfrom(1024)
17         message = data.decode()
18         print("Received:", message)
19         receivedPackets.append(message)
20
21     except timeout:
22         print("No more packets received.")
23         break
24
25 numbers = []
26
27 for packet in receivedPackets:
28     number = int(packet.split("-")[1])
29     numbers.append(number)
30
31 missing = []
32
33 for i in range(1, 201):
34     if i not in numbers:
35         missing.append(i)
36
37 outOfOrder = 0
38
39 for i in range(1, len(numbers)):
40     if numbers[i] < numbers[i - 1]:
41         outOfOrder += 1
42
43 print("Total packets received:", len(numbers))
44 print("Number of missing packets:", len(missing))
45 print("Number of out-of-order packets:", outOfOrder)
46
47 serverSocket.close()
```

Figure 15: UDP server program

Next, the UDP client program was implemented as shown in Figure(16) below. A UDP socket was created and configured to send packets to the server using the `sendto()` function. The client generated and transmitted 200 numbered packets in the format Packet-1 to Packet-200. In addition, a small delay using `time.sleep(0.1)` was added between packet transmissions to make the packet transfer process easier to observe during execution and testing.

A screenshot of a Visual Studio Code editor window. The editor has two tabs open: 'client.py' and 'server.py'. The 'client.py' tab is active, showing a Python script. The script starts with imports for 'socket' and 'time'. It then defines 'serverName' using 'gethostname()' and 'serverPort' as 7200, with a comment explaining the port calculation: '# SID: 1230199 -> 7001 + 199 = 7200'. A UDP socket is created with 'socket(AF_INET, SOCK_DGRAM)'. A loop from 1 to 200 generates messages in the format 'Packet-' followed by the packet number. Each message is encoded and sent to the server using 'clientSocket.sendto()'. A print statement shows the sent message, and a 0.1-second sleep is added between sends. Finally, the socket is closed.

```
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7200      # SID: 1230199 -> 7001 + 199 = 7200
6
7 clientSocket = socket(AF_INET, SOCK_DGRAM)
8
9 for i in range (1,201):
10     message = "Packet-" + str(i)
11     clientSocket.sendto(message.encode(), (serverName, serverPort))
12     print("Packet Sent: ", message)
13     time.sleep(0.1)
14
15
16 clientSocket.close()
```

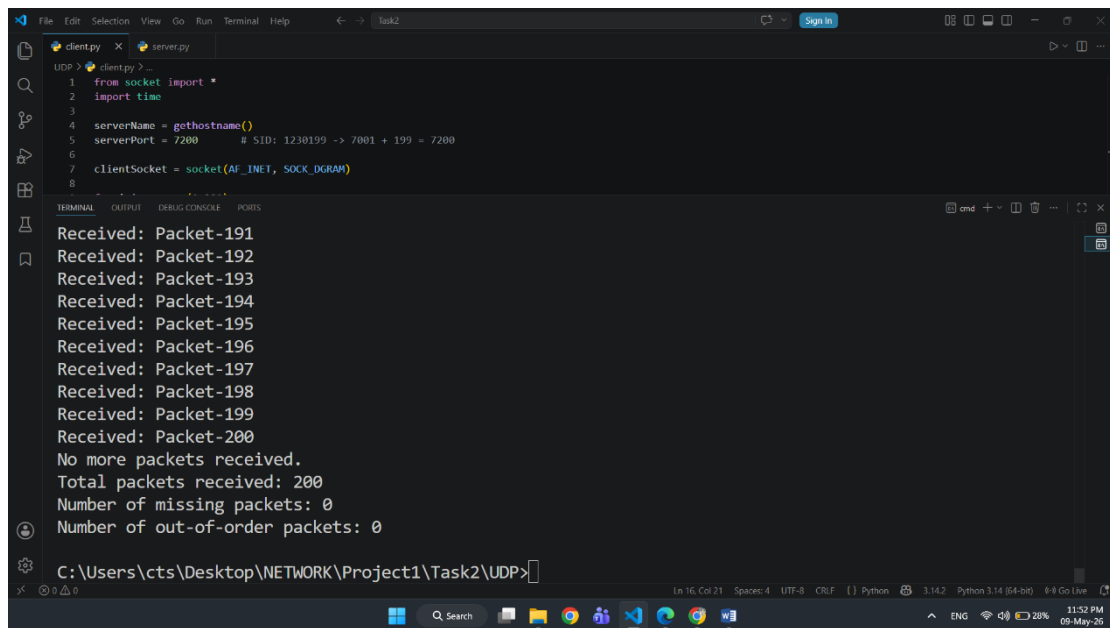
Figure 16: UDP client program

For testing purposes, the programs were executed using separate terminal windows in Visual Studio Code. Different execution scenarios were tested by changing the startup order of the client and server programs. The outputs of both applications were monitored through the terminal windows, and screenshots were captured showing the transmitted packets, received packets, and the final packet analysis statistics.

Results

Client after Server

In the first test case, the UDP server was started before the client program. As shown in Figures (17) and (18), the client successfully transmitted all 200 packets, and the server received all packets correctly without any missing or out-of-order packets. This indicates that the server was fully ready to receive packets before the transmission process began, allowing all packets to arrive successfully.

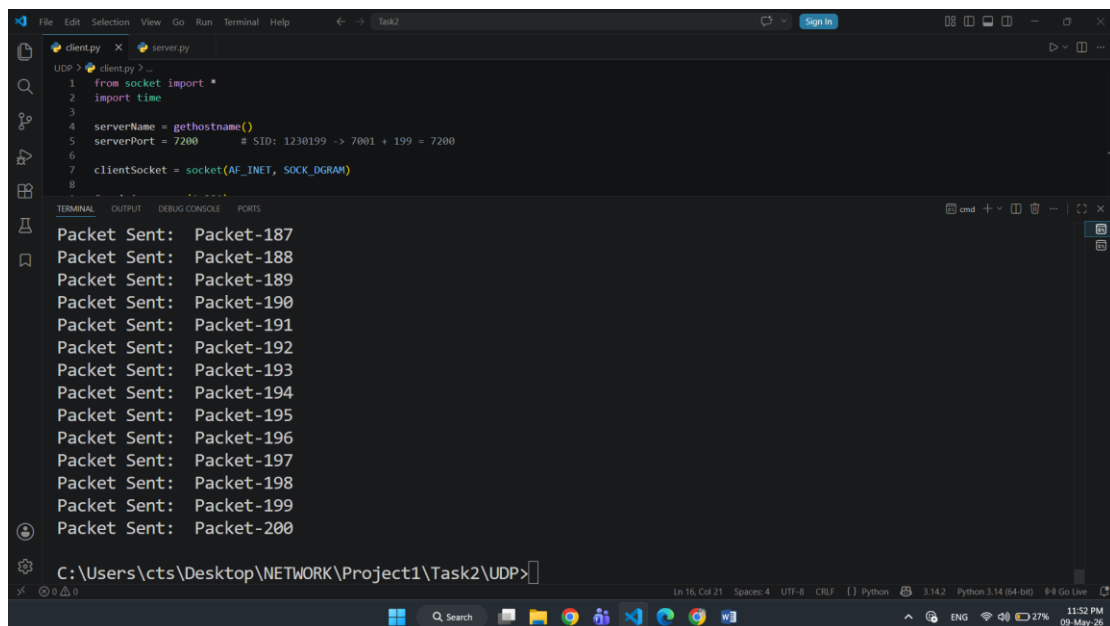


```
UDP > client.py > ...
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7200 # SID: 1230199 -> 7001 + 199 = 7200
6
7 clientSocket = socket(AF_INET, SOCK_DGRAM)
8

Received: Packet-191
Received: Packet-192
Received: Packet-193
Received: Packet-194
Received: Packet-195
Received: Packet-196
Received: Packet-197
Received: Packet-198
Received: Packet-199
Received: Packet-200
No more packets received.
Total packets received: 200
Number of missing packets: 0
Number of out-of-order packets: 0

C:\Users\cts\Desktop\NETWORK\Project1\Task2\UDP>
```

Figure 17: Output of the UDP server program – Client after Server



```
UDP > client.py > ...
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7200 # SID: 1230199 -> 7001 + 199 = 7200
6
7 clientSocket = socket(AF_INET, SOCK_DGRAM)
8

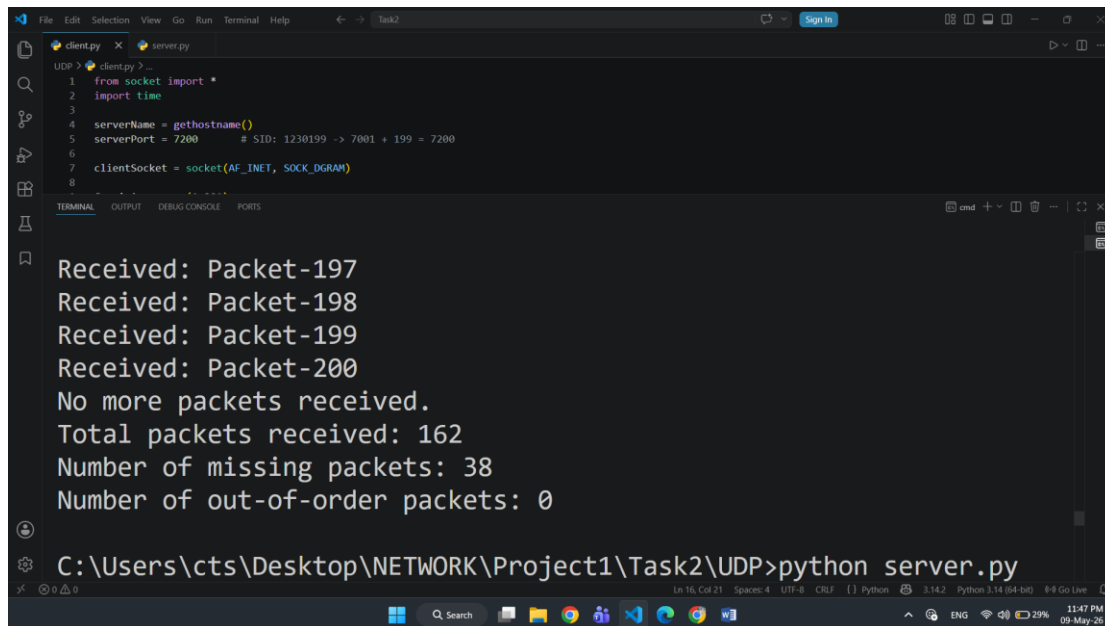
Packet Sent: Packet-187
Packet Sent: Packet-188
Packet Sent: Packet-189
Packet Sent: Packet-190
Packet Sent: Packet-191
Packet Sent: Packet-192
Packet Sent: Packet-193
Packet Sent: Packet-194
Packet Sent: Packet-195
Packet Sent: Packet-196
Packet Sent: Packet-197
Packet Sent: Packet-198
Packet Sent: Packet-199
Packet Sent: Packet-200

C:\Users\cts\Desktop\NETWORK\Project1\Task2\UDP>
```

Figure 18: Output of UDP client program – Client after Server

Client before Server

In the second test case, the client program was started before the server. As shown in Figures (19) and (20), the server received only 162 packets and detected 38 missing packets, while no out-of-order packets were observed. The missing packets corresponded to the packets transmitted before the server became ready to receive data, demonstrating the connectionless behavior of UDP communication.

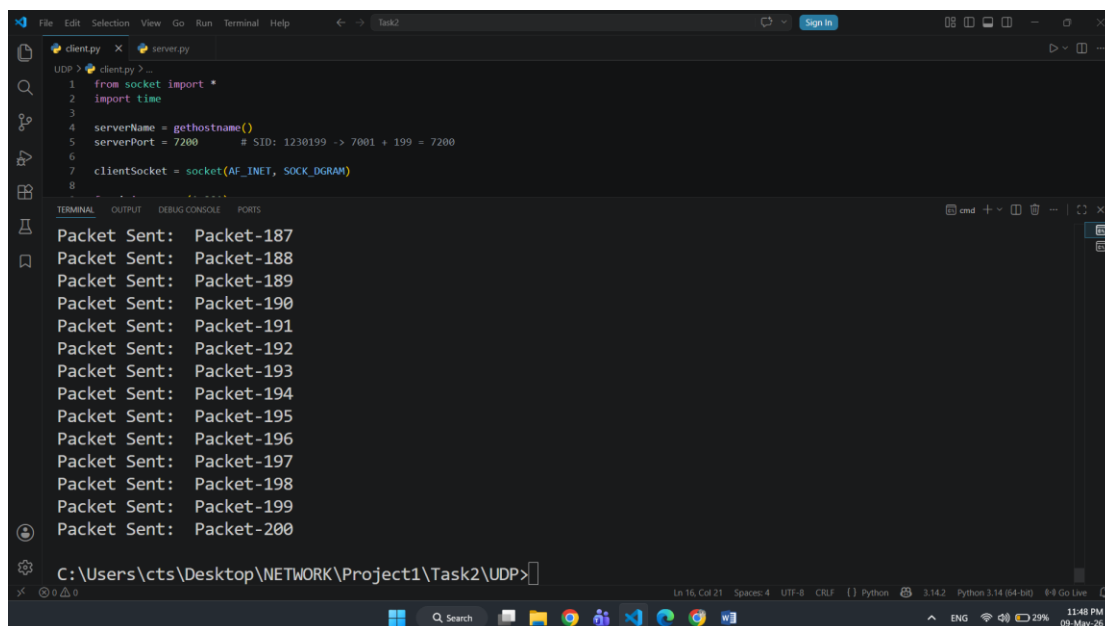


```
client.py x server.py
UDP > client.py > ...
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7200 # SID: 1230199 -> 7001 + 199 = 7200
6
7 clientSocket = socket(AF_INET, SOCK_DGRAM)
8

TERMINAL OUTPUT DEBUG CONSOLE PORTS
Received: Packet-197
Received: Packet-198
Received: Packet-199
Received: Packet-200
No more packets received.
Total packets received: 162
Number of missing packets: 38
Number of out-of-order packets: 0

C:\Users\cts\Desktop\NETWORK\Project1\Task2\UDP>python server.py
```

Figure 19: Output of UDP server program – Client before Server



```
client.py x server.py
UDP > client.py > ...
1 from socket import *
2 import time
3
4 serverName = gethostname()
5 serverPort = 7200 # SID: 1230199 -> 7001 + 199 = 7200
6
7 clientSocket = socket(AF_INET, SOCK_DGRAM)
8

TERMINAL OUTPUT DEBUG CONSOLE PORTS
Packet Sent: Packet-187
Packet Sent: Packet-188
Packet Sent: Packet-189
Packet Sent: Packet-190
Packet Sent: Packet-191
Packet Sent: Packet-192
Packet Sent: Packet-193
Packet Sent: Packet-194
Packet Sent: Packet-195
Packet Sent: Packet-196
Packet Sent: Packet-197
Packet Sent: Packet-198
Packet Sent: Packet-199
Packet Sent: Packet-200

C:\Users\cts\Desktop\NETWORK\Project1\Task2\UDP>
```

Figure 20: Output of UDP client program – Client before Server

Discussion and Analysis

The obtained results clearly highlight the major differences between TCP and UDP communication. TCP demonstrated reliable communication by successfully delivering all transmitted messages in the correct order without any loss. This reliability is achieved because TCP is a connection-oriented protocol that establishes a connection between the client and the server before data transmission begins. In addition, TCP uses acknowledgment messages, sequencing, and retransmission mechanisms to ensure that packets arrive correctly and in order. Therefore, when the server was unavailable, the TCP client could not start communication and immediately generated a connection error instead of sending data that could potentially be lost.

On the other hand, UDP showed a less reliable behavior because it is a connectionless protocol that transmits packets without establishing a connection or verifying successful delivery. As a result, packets transmitted before the server became ready were lost permanently, leading to missing packets in the second UDP test case. Unlike TCP, UDP does not provide acknowledgments, retransmission, flow control, or packet ordering mechanisms, which makes communication faster and simpler but less reliable. However, no out-of-order packets were observed during the UDP tests because communication was performed locally using localhost, where packet transmission was stable and very fast.

Overall, TCP was found to be more reliable than UDP because it guarantees ordered and error-checked delivery through connection establishment, acknowledgments, and retransmission mechanisms. UDP, on the other hand, provides faster communication with lower overhead but does not guarantee reliability, packet ordering, or successful delivery of data.

Task 3 – Simple File-Sharing Web Server using Socket Programming

Theoretical Background

A web server is a system that handles requests coming from clients such as web browsers and responds by delivering different types of resources like HTML pages, CSS files, images, and text files. The communication between the client and the server is based on the HTTP (HyperText Transfer Protocol), which defines how messages are structured and exchanged over the network. In this process, the client sends an HTTP request that includes a method such as GET, a requested path, and optional data, while the server processes the request and returns an HTTP response containing a status code, headers, and the requested content [3] [4].

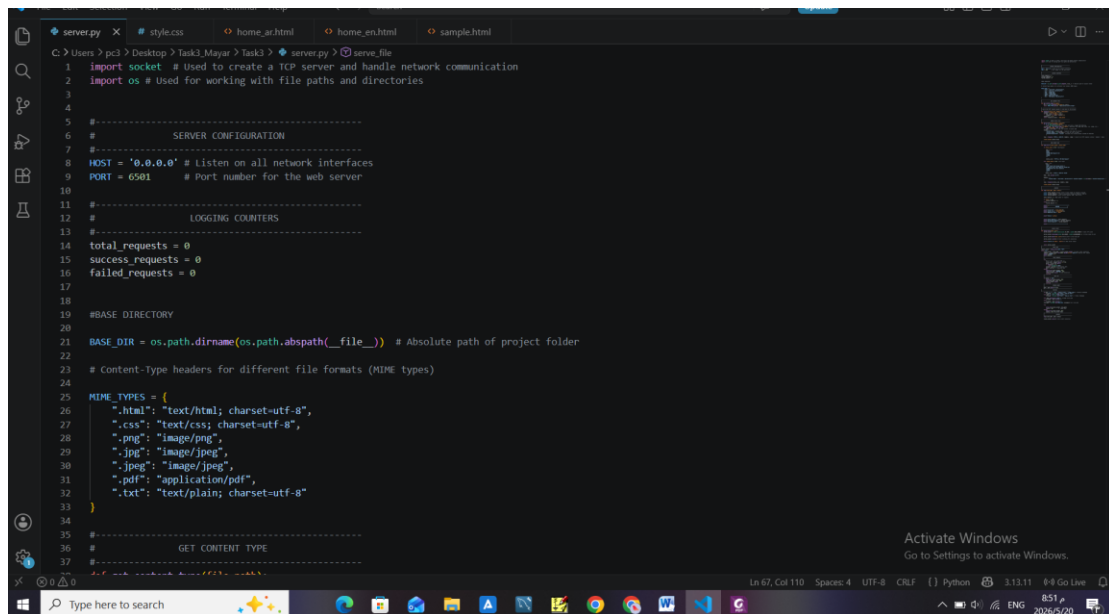
The communication between the client and server is built on TCP (Transmission Control Protocol), which is a connection-oriented protocol that ensures reliable data transmission. TCP establishes a connection between both sides before any data is exchanged through a process called the three-way handshake, which guarantees that both the client and server are ready for communication. It also ensures that data is delivered in order, without loss or duplication, making it suitable for web applications where accuracy and reliability are important. In this project, TCP sockets were used to create a direct and continuous connection between the client and the server. [4]

The server is also capable of serving different types of files depending on the request, including images, text files, and web pages. Each file is identified by its extension, and the appropriate Content-Type is sent to the browser so it can properly interpret and display the content.

Procedure

Web Server

This section explains the logic and implementation of the Python web server. The server uses low-level socket programming to receive HTTP requests, parse them, generate responses.

A screenshot of a code editor showing a Python script for a web server. The script includes imports for socket and os, followed by server configuration variables like HOST, PORT, and BASE_DIR. It also defines logging counters and a MIME types dictionary. The code is written in a dark-themed editor with a Windows taskbar visible at the bottom.

```
1 import socket # Used to create a TCP server and handle network communication
2 import os # Used for working with file paths and directories
3
4
5 #-----
6 # SERVER CONFIGURATION
7 #-----
8 HOST = '0.0.0.0' # listen on all network interfaces
9 PORT = 6501 # Port number for the web server
10
11 #-----
12 # LOGGING COUNTERS
13 #-----
14 total_requests = 0
15 success_requests = 0
16 failed_requests = 0
17
18 #BASE DIRECTORY
19
20
21 BASE_DIR = os.path.dirname(os.path.abspath(__file__)) # Absolute path of project folder
22
23 # Content-Type headers for different file formats (MIME types)
24
25 MIME_TYPES = {
26     ".html": "text/html; charset=utf-8",
27     ".css": "text/css; charset=utf-8",
28     ".png": "image/png",
29     ".jpg": "image/jpeg",
30     ".jpeg": "image/jpeg",
31     ".pdf": "application/pdf",
32     ".txt": "text/plain; charset=utf-8"
33 }
34
35 #-----
36 # GET CONTENT TYPE
37 #-----
```

Figure 21: Web Server program – Imports and Global Variables

Several Python modules required for building the web server were imported:

- Socket: is used to create a TCP server and handle network communication between client and server, allowing HTTP message exchange over a specific port.
- Os: is used to handle file paths, locate the project directory, and build paths to static files such as HTML, CSS, images, and text files.

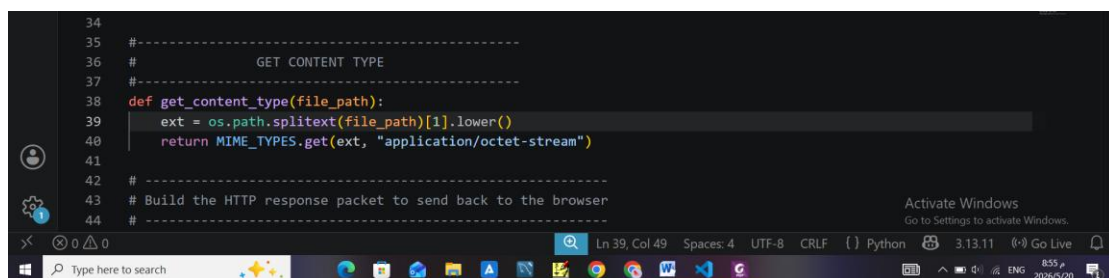
Global configuration variables were defined:

- HOST = '0.0.0.0': server listens on all network interfaces, allowing external devices to connect.
- PORT = 6501: defines the port where the server listens for incoming requests.
- BASE_DIR: absolute path of the project directory used to locate files safely.

- Logging counters are initialized:
 - total_requests: counts all requests received.
 - success_requests: counts successful responses.
 - failed_requests: counts failed requests.
- MIME_TYPES maps file extensions to correct Content-Type headers so the browser can correctly display different file types.

Then some functions were defined:

get_content_type() function:



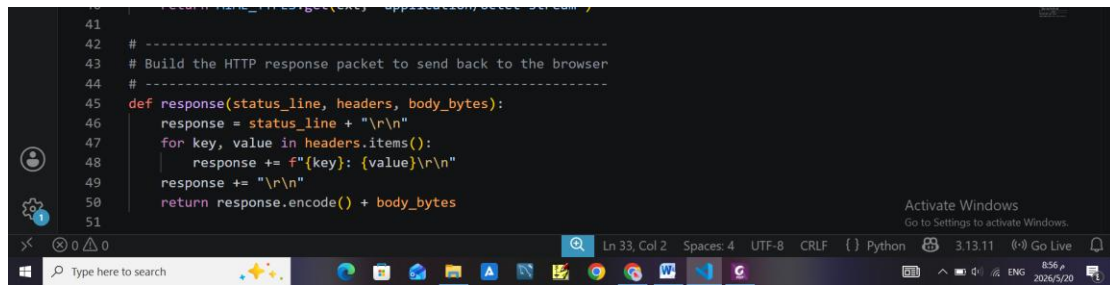
```

34
35 #-----
36 #           GET CONTENT TYPE
37 #-----
38 def get_content_type(file_path):
39     ext = os.path.splitext(file_path)[1].lower()
40     return MIME_TYPES.get(ext, "application/octet-stream")
41
42 # -----
43 # Build the HTTP response packet to send back to the browser
44 # -----
  
```

Figure 22: Web Server program – get_content_Type() function

This function `get_content_type(file_path)` is used to determine the type of a file based on its extension so the server can handle it correctly. It takes the file path as input and uses `os.path.splitext(file_path)[1]` to split the file name and extract only the extension such as `.html`, `.css`, `.js`, `.png`, `.jpg`, `.txt`, and other similar types. Then the extension is converted to lowercase using `.lower()` to ensure accurate comparison regardless of letter case. After that, the function checks this extension inside the `MIME_TYPES` dictionary to find the corresponding content type like `text/html`, `text/css`, or image types. If the extension exists, it returns the correct MIME type; otherwise, it returns the default value `"application/octet-stream"` which means an unknown or unsupported file type.

response() function:

A screenshot of a Python IDE window showing a code editor with a dark background. The code defines a function named 'response' that takes three arguments: 'status_line', 'headers', and 'body_bytes'. The function starts by creating a string 'response' with the 'status_line' followed by a carriage return and a newline character. It then iterates over the 'headers' dictionary, appending each key-value pair to the 'response' string in the format 'key: value' followed by a carriage return and a newline character. After the loop, it appends another carriage return and a newline character. Finally, it returns the 'response' string encoded as bytes, concatenated with 'body_bytes'. The IDE interface includes a search bar, a taskbar with various application icons, and a status bar at the bottom showing file encoding (UTF-8) and line/column numbers (Ln 33, Col 2).

```
41 # -----  
42 # Build the HTTP response packet to send back to the browser  
43 # -----  
44  
45 def response(status_line, headers, body_bytes):  
46     response = status_line + "\r\n"  
47     for key, value in headers.items():  
48         response += f"{key}: {value}\r\n"  
49     response += "\r\n"  
50     return response.encode() + body_bytes  
51
```

Figure 23: Web Server program – response() function

This function builds a complete **HTTP response message** by combining three main parts: the status line, headers, and body. It is used to ensure that the server sends data in a proper HTTP format so the browser can understand and display it correctly, whether the content is HTML, CSS, images, or any other type.

The function starts by creating a string that contains the **status line**, followed by `\r\n` to mark the end of the line. Then, it loops through the headers dictionary and appends each header in the format `key: value\r\n`. After adding all headers, it adds an extra blank line (`\r\n`) to indicate the end of the header section, as required by the HTTP protocol. Finally, the function encodes the response headers into bytes and appends the body in byte format to create the final complete HTTP response message that is sent to the client.

server_file() function:

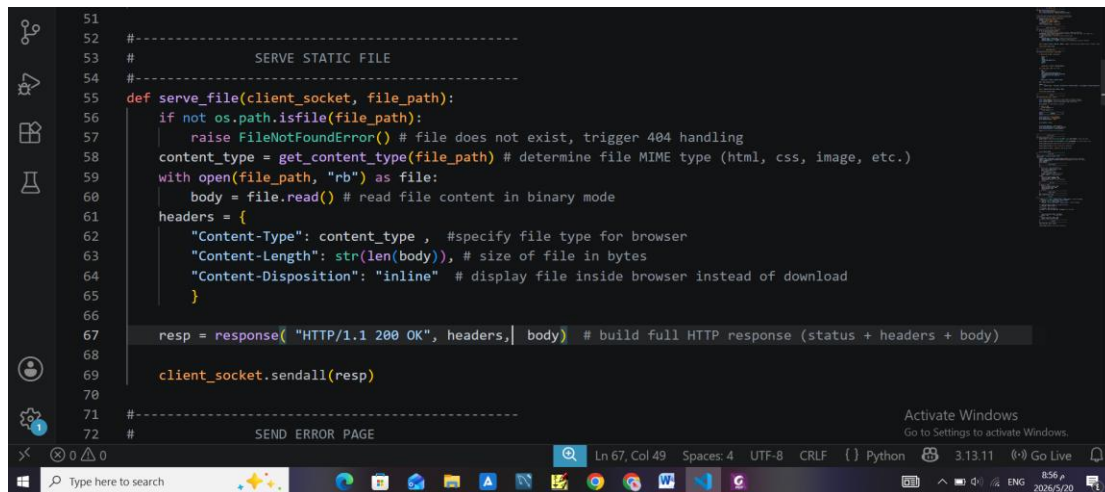
A screenshot of a code editor with a dark theme. The editor shows a Python function named `serve_file` defined between lines 55 and 72. The function takes `client_socket` and `file_path` as arguments. It checks if the file exists using `os.path.isfile`. If not, it raises `FileNotFoundError`. If it exists, it gets the content type using `get_content_type`, opens the file in binary mode (`'rb'`), reads the content into `body`, and creates a dictionary of HTTP headers. The headers include `Content-Type`, `Content-Length` (calculated as `len(body)`), and `Content-Disposition` set to `"inline"`. A full HTTP response is built using `response()` with status `"HTTP/1.1 200 OK"`, the headers, and the body. Finally, the response is sent to the client using `client_socket.sendall`. The code is surrounded by comments indicating it's for serving static files and sending error pages. The editor's status bar at the bottom shows the cursor is at line 67, column 49, with 4 spaces, UTF-8 encoding, and CRLF line endings. The taskbar at the very bottom shows various application icons and the system clock indicating 8:56 PM on 2026/5/20.

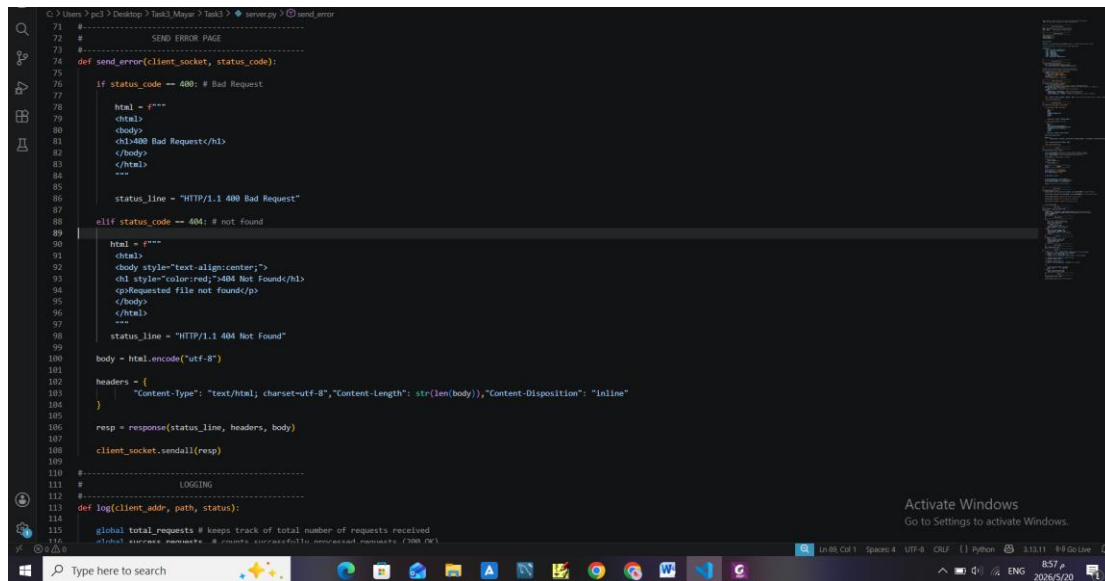
Figure 24: Web Server program – `server_file()` function

This function `serve_file(client_socket, file_path)` is responsible for serving a file from the server to the client. First, it receives the file path and checks whether the file actually exists using `os.path.isfile(file_path)`. If the file does not exist, it raises a `FileNotFoundError` to indicate an error. If the file exists, the function determines its type by calling `get_content_type(file_path)`, which identifies the correct MIME type based on the file extension. Then, the file is opened in **binary mode (rb)** to ensure that all types of files, including images, are handled correctly without data corruption. The file content is read and stored in the variable `body`.

After that, the function creates the HTTP headers, including `Content-Type` to specify the file type, `Content-Length` to indicate the size of the file, and `Content-Disposition` set to `"inline"` to control how the browser displays the file directly inside the page instead of forcing it to download. Next, it builds a complete HTTP response by calling the `response()` function with a success status line `"HTTP/1.1 200 OK"`, the headers, and the body.

Finally, the complete response is sent to the client using `client_socket.sendall(resp)`, which delivers the file successfully to the browser.

send_error() function:

The image is a screenshot of a code editor window. The editor has a dark theme. On the left side, there is a sidebar with icons for file explorer, search, and other IDE features. The main area displays a Python script. The script defines a function named `send_error(client_socket, status_code)`. Inside this function, there are two conditional branches based on the `status_code` parameter. The first branch is for `status_code == 400`, which creates an HTML response for a "Bad Request" error. The second branch is for `status_code == 404`, which creates an HTML response for a "Not Found" error. Both responses include a status line, headers (Content-Type, Content-Length, Content-Disposition), and a body containing the error message. The response is then encoded to bytes and sent to the client using `client_socket.sendall(resp)`. At the bottom of the editor, a Windows taskbar is visible with various application icons and system tray icons. The system clock shows the date as 20/6/2020 and the time as 8:57 PM. An "Activate Windows" watermark is also present in the bottom right corner of the editor window.

```
71 #-----  
72 # SEND ERROR PAGE  
73 #-----  
74 def send_error(client_socket, status_code):  
75     if status_code == 400: # Bad Request  
76  
77         html = """  
78         <html>  
79         <html>  
80         <body>  
81         <h1>400 Bad Request</h1>  
82         </body>  
83         </html>  
84         """  
85  
86         status_line = "HTTP/1.1 400 Bad Request"  
87  
88     elif status_code == 404: # not found  
89  
90         html = """  
91         <html>  
92         <body style="text-align:center;">  
93         <h1 style="color:red;">404 Not Found</h1>  
94         <p>Requested file not found</p>  
95         </body>  
96         </html>  
97         """  
98  
99         status_line = "HTTP/1.1 404 Not Found"  
100  
101         body = html.encode("utf-8")  
102  
103         headers = {  
104             "Content-Type": "text/html; charset=utf-8", "Content-Length": str(len(body)), "Content-Disposition": "inline"  
105         }  
106  
107         resp = response(status_line, headers, body)  
108  
109         client_socket.sendall(resp)  
110  
111     #-----  
112     # LOGGING  
113     #-----  
114     def log(client_addr, path, status):  
115         global total_requests # keeps track of total number of requests received  
116         global server_requests # counts requests to server's resources (3000 PM)
```

Figure 25: Web Server program – send_error() function

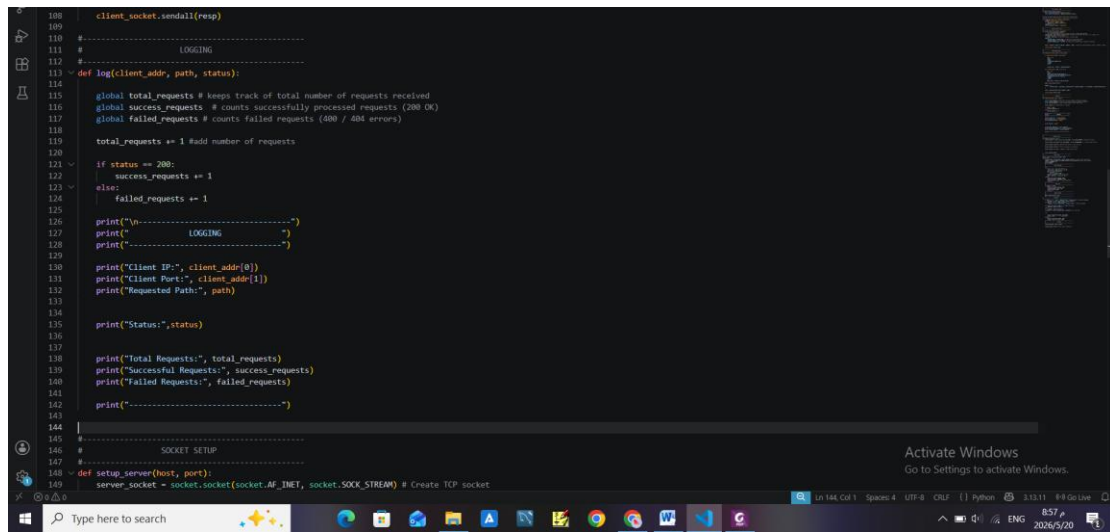
This function `send_error(client_socket, status_code)` is used to send an HTTP error response to the client when something goes wrong, such as a bad request or a missing file. It takes the client socket and the error status code as inputs.

First, the function checks the value of `status_code`. If it is 400, it creates an HTML page that displays a **400 Bad Request** error message and sets the corresponding HTTP status line "HTTP/1.1 400 Bad Request". If the status code is 404, it generates a **404 Not Found** HTML page with a message indicating that the requested file was not found, and sets the status line to "HTTP/1.1 404 Not Found".

After defining the HTML content, it is encoded into bytes using UTF-8. Then, the function builds the HTTP headers, including Content-Type set to text/html; charset=utf-8 and Content-Length which represents the size of the response body.

Next, the function creates the full HTTP response by calling the `response()` function with the status line, headers, and body. Finally, the complete error response is sent to the client using `client_socket.sendall(resp)` so the browser displays the appropriate error page.

log() function:

A screenshot of a code editor window showing a Python script. The script is a web server program. The log() function is defined at line 113. It takes three arguments: client_addr, path, and status. It uses global variables total_requests, success_requests, and failed_requests to keep track of server statistics. The function increments total_requests by 1 for every request. It then checks the status: if status is 200, it increments success_requests; otherwise, it increments failed_requests. After updating the counters, it prints a formatted log section that includes the client's IP address and port, the requested file path, and the response status. It also displays the total number of requests along with how many were successful and how many failed. The code is as follows:

```
100 client_socket.sendall(resp)
101
102 #-----
103 # LOGGING
104 #-----
105 def log(client_addr, path, status):
106     global total_requests # keeps track of total number of requests received
107     global success_requests # counts successfully processed requests (200 OK)
108     global failed_requests # counts failed requests (400 / 404 errors)
109     total_requests += 1 #add number of requests
110
111     if status == 200:
112         success_requests += 1
113     else:
114         failed_requests += 1
115
116     print("\n-----")
117     print(" LOGGING ")
118     print("-----")
119     print("Client IP:", client_addr[0])
120     print("Client Port:", client_addr[1])
121     print("Requested Path:", path)
122
123     print("Status:", status)
124
125     print("Total Requests:", total_requests)
126     print("Successful Requests:", success_requests)
127     print("Failed Requests:", failed_requests)
128     print("-----")
129
130 #-----
131 # SOCKET SETUP
132 #-----
133 def setup_server(host, port):
134     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Create TCP socket
```

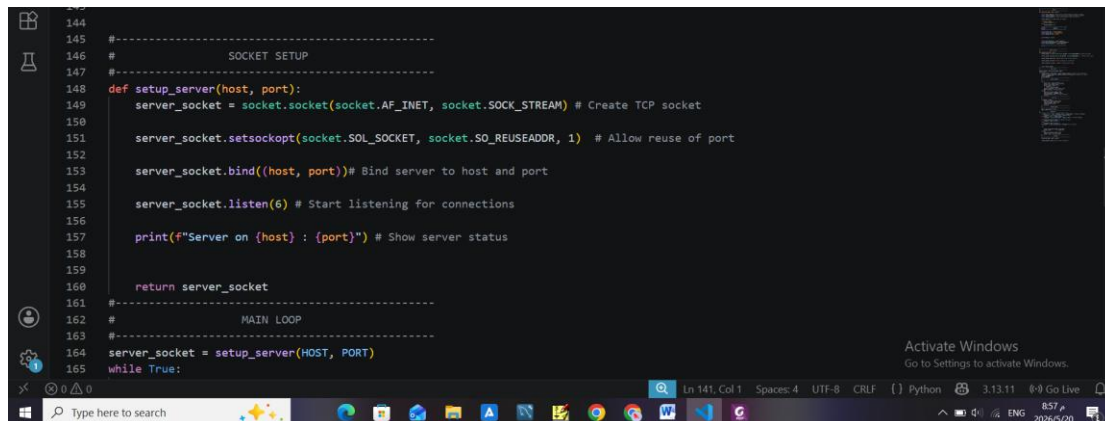
Figure 26: Web Server program – log() function

This function `log(client_addr, path, status)` is used to record and display information about each request received by the server. It helps in monitoring server activity and tracking how many requests are successful or failed.

First, the function uses global variables `total_requests`, `success_requests`, and `failed_requests` to keep track of overall server statistics. Every time a request is received, `total_requests` is increased by one. Then, based on the HTTP status, the function checks whether the request was successful or not. If the status is 200, it increases `success_requests`; otherwise, it increases `failed_requests`.

After updating the counters, the function prints a formatted log section that includes the client's IP address and port, the requested file path, and the response status. It also displays the total number of requests along with how many were successful and how many failed. This makes it easier to debug and monitor server performance.

setup_server() function:



```
144 #-----  
145 #          SOCKET SETUP  
146 #-----  
147 #  
148 def setup_server(host, port):  
149     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Create TCP socket  
150  
151     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # Allow reuse of port  
152  
153     server_socket.bind((host, port)) # Bind server to host and port  
154  
155     server_socket.listen(6) # Start listening for connections  
156  
157     print(f"Server on {host} : {port}") # Show server status  
158  
159  
160     return server_socket  
161 #-----  
162 #          MAIN LOOP  
163 #-----  
164 server_socket = setup_server(HOST, PORT)  
165 while True:
```

Figure 27: Web Server program – setup_server() function

This function is used to fully initialize and prepare a TCP server before it starts accepting any client connections. First, a new socket is created using the socket library with IPv4 addressing (AF_INET) and TCP protocol (SOCK_STREAM). This ensures that communication is reliable and connection-oriented, meaning data is delivered in order without loss.

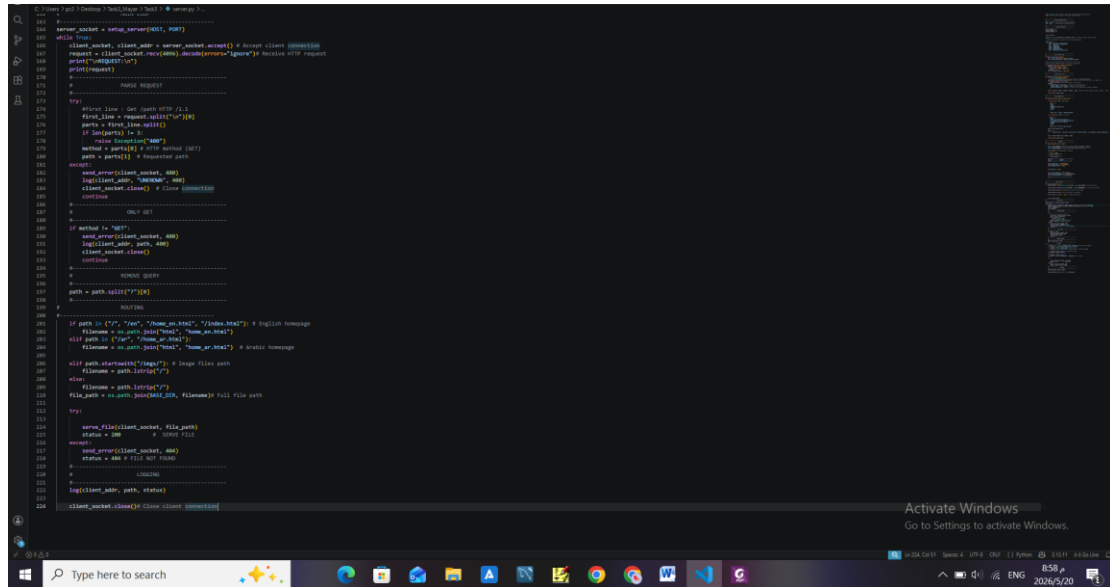
Next, the setsockopt function is used to enable the SO_REUSEADDR option. This is an important feature that allows the server to reuse the same port immediately after restarting, without waiting for the operating system to fully release it. This helps avoid the common “port already in use” error.

After that, the bind function is used to associate the server with a specific host address and port number. This defines where the server will listen for incoming connections.

Then, the listen function is called with a backlog value of 6, which specifies the maximum number of incoming connection requests that can be queued while waiting to be accepted by the server.

Finally, the function prints a message indicating that the server is running successfully on the given host and port, and returns the socket object so it can be used later in the main program for handling client connections and requests.

Main Server Loop:

A screenshot of a code editor displaying a Python web server program. The code is written in a dark-themed editor with line numbers on the left. The program implements a simple HTTP server. It starts with a `while True` loop that listens for incoming connections using `server\_socket.accept()`. When a connection is accepted, it prints the client's address and the received request. The request is then split into lines, and the first line is used to determine the HTTP method and the requested path. The code handles GET requests by reading a file from the local filesystem and sending its contents to the client. It also handles POST requests by receiving data from the client and printing it. The server socket is closed after each request, and the loop continues to listen for the next connection. The Windows taskbar is visible at the bottom of the screen, showing the time as 8:58 PM on 2024/5/20.

```
1 # Import the necessary modules
2 import socket
3 import sys
4 import os
5
6 # Define the host and port
7 HOST = '0.0.0.0'
8 PORT = 8080
9
10 # Create a server socket
11 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12
13 # Bind the server socket to the host and port
14 server_socket.bind((HOST, PORT))
15
16 # Listen for incoming connections
17 server_socket.listen(1)
18
19 # Main server loop
20 while True:
21     # Accept a new connection
22     client_socket, client_addr = server_socket.accept()
23     print(f'Connection from {client_addr}')
24     request = client_socket.recv(4096).decode('utf-8')
25     print(request)
26
27     # Parse the request
28     lines = request.split('\n')
29     if len(lines) < 1:
30         continue
31     method, path = lines[0].split(' ')
32     path = path.lstrip('/')
33
34     # Handle GET request
35     if method == 'GET':
36         # Read the file
37         filename = os.path.join('static', path)
38         if not os.path.exists(filename):
39             continue
40         with open(filename, 'rb') as f:
41             data = f.read()
42         client_socket.send(data)
43         client_socket.close()
44     else:
45         # Handle POST request
46         # Receive the data
47         data = client_socket.recv(4096).decode('utf-8')
48         print(data)
49         client_socket.close()
50
51 # Close the server socket
52 server_socket.close()
```

Figure 28: Web Server program – Main Server Loop

This loop represents the core of the server's operation, ensuring that the server runs continuously without stopping by using an infinite loop (while True). The main purpose of this loop is to keep the server in a constant listening state so that it can handle multiple client requests over time instead of terminating after processing a single request.

At the beginning, the server is already initialized outside the loop using `setup_server(HOST, PORT)`, where `HOST` and `PORT` define the network interface and port number on which the server listens for incoming connections.

In each iteration, the server waits for a new incoming client connection using the `accept` method. This method establishes a connection between the server and the client and returns both the client's address information (IP and port) and a dedicated socket for communication.

Once the connection is established, the server receives data sent by the client using the `recv` function and converts it into a readable HTTP request format. The request is then printed for debugging and monitoring purposes.

Next comes the request parsing stage, where the first line of the HTTP request is extracted because it contains the HTTP method and the requested path. The line is split and validated to ensure it follows the correct HTTP structure (three parts). If the request

format is invalid, a 400 Bad Request error is sent, and the connection is immediately closed.

If the request is valid, the server checks whether the HTTP method is GET. If any other method is used, the request is rejected with a 400 error as well.

After that, the server removes any query parameters (Query String) such as `?id=1`, since they are not required for basic routing logic.

Then the routing phase begins, where the requested path is mapped to the appropriate file. The server now supports serving both HTML pages and static files such as images. For example, paths like `/`, `/en`, `/home_en.html`, and `/index.html` are mapped to the English homepage, while `/ar` and `/home_ar.html` are mapped to the Arabic homepage. For image files (paths starting with `/imgs/`), the path is directly converted into a file system path. For all other requests, the server treats them as direct file paths relative to a defined base directory (`BASE_DIR`), ensuring proper file location resolution.

After determining the full file path, the server attempts to send the file to the client. If successful, a 200 OK status is returned; otherwise, a 404 Not Found error is sent.

Finally, the request is logged with the client address, requested path, and status code. The client connection is then closed, and the loop returns to its initial state to wait for the next incoming request.

HTML

HTML files were created to implement the required main pages of the file-sharing web server. The project required an English homepage and an identical Arabic version, so two HTML pages were designed: home_en.html and home_ar.html. These pages include the title “ENCS3320 File Sharing Server”, a short welcome message with words in different colors, a list of available shared files, and download links for each file. In addition, the required team members’ section was implemented using a table that shows each student’s name, ID, department, completed credit hours, and student ID card image. The HTML structure also includes navigation between the English and Arabic versions, which makes the web server easier to use and matches the project requirement for bilingual pages.

CSS

A separate CSS stylesheet was created to enhance the visual appearance and organization of the web pages. The stylesheet was used to design the layout, colors, fonts, tables, buttons, and image presentation, in addition to improving the overall user interface and user experience of the file-sharing web server. Figures (29) and (30) show the final appearance of the web server pages after applying the HTML structure and CSS styling.

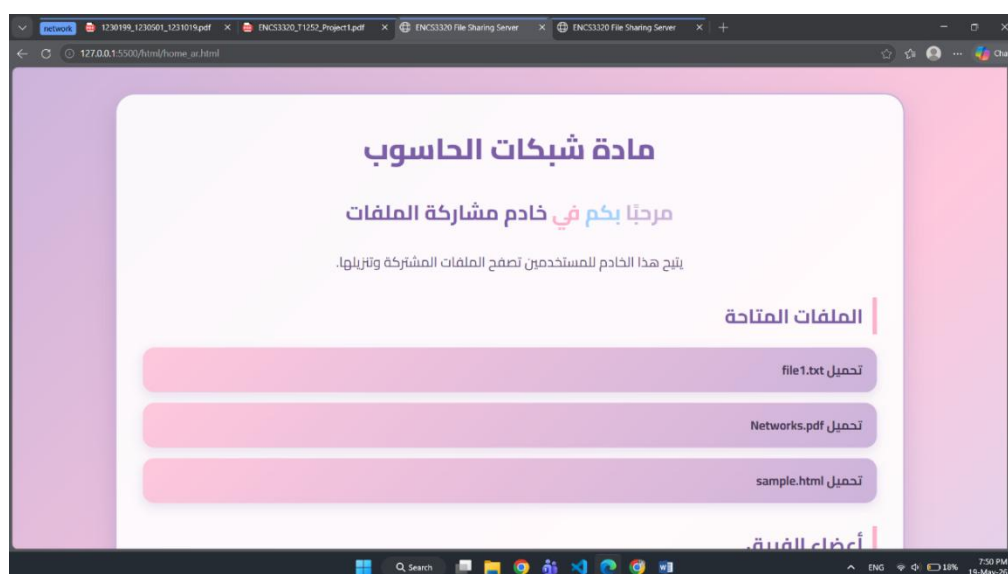


Figure 29: Arabic Homepage of the File-Sharing Web Server (1)

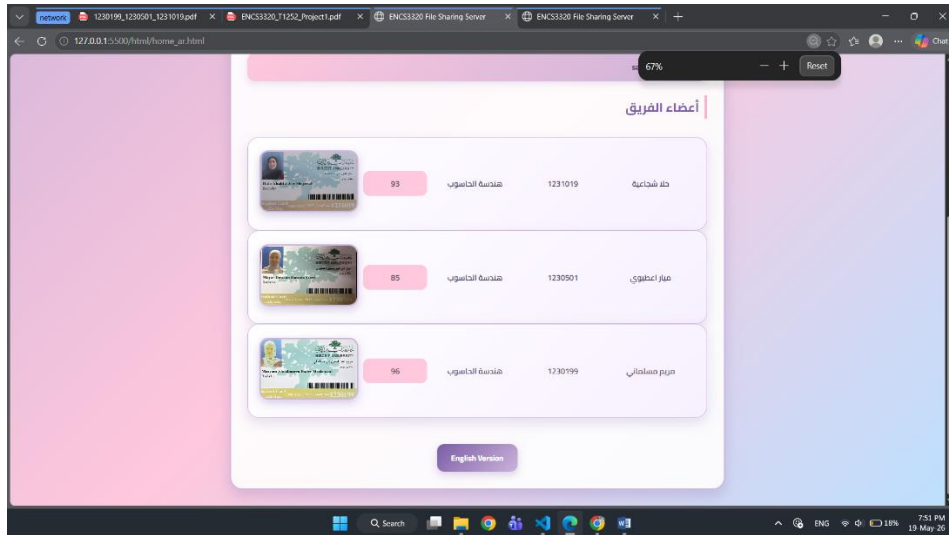


Figure 30: Arabic Homepage of the File-Sharing Web Server (2)

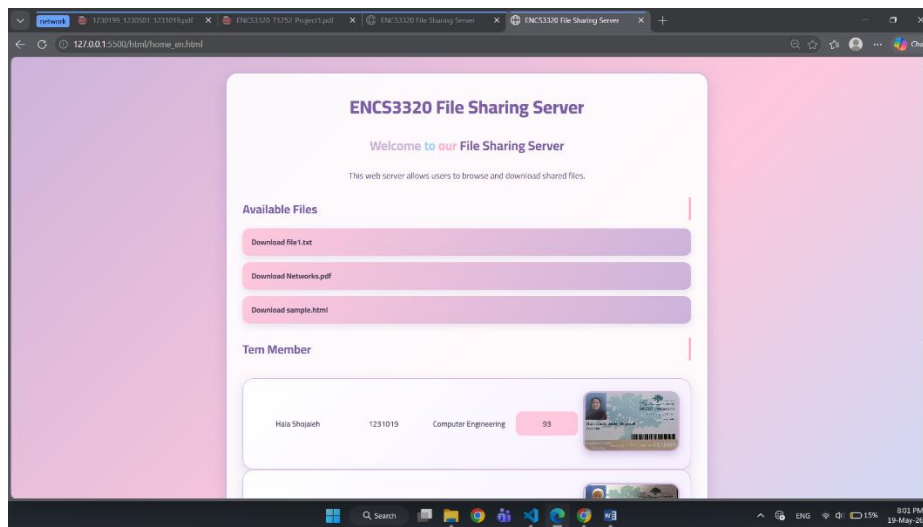


Figure 31: English Homepage of the File-Sharing Web Server (1)

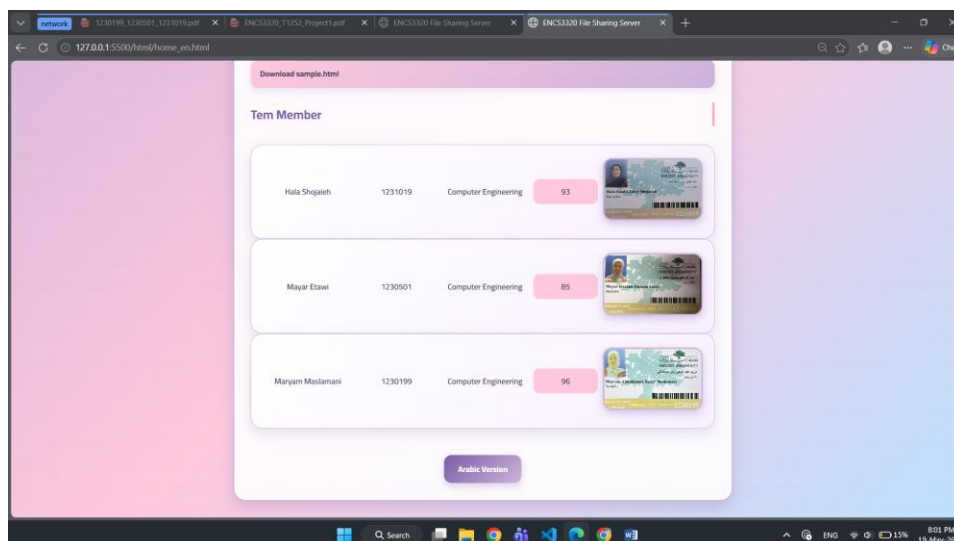


Figure 32: English Homepage of the File-Sharing Web Server (2)

Results, Discussion and Analysis

Server Startup

This test verifies that the web server starts correctly and listens on port 6501.

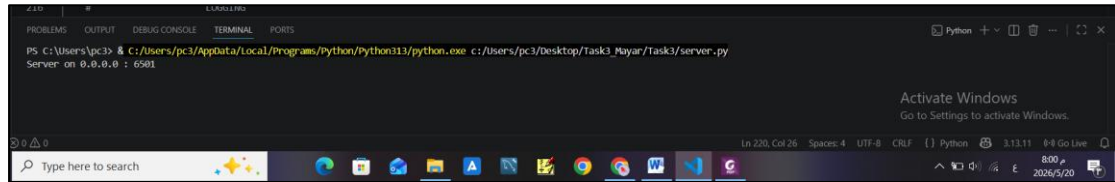


Figure 33: Server startup

English Page Tests

The following tests ensure that the English main page is correctly served for different valid URLs.

- Accessing English Page:

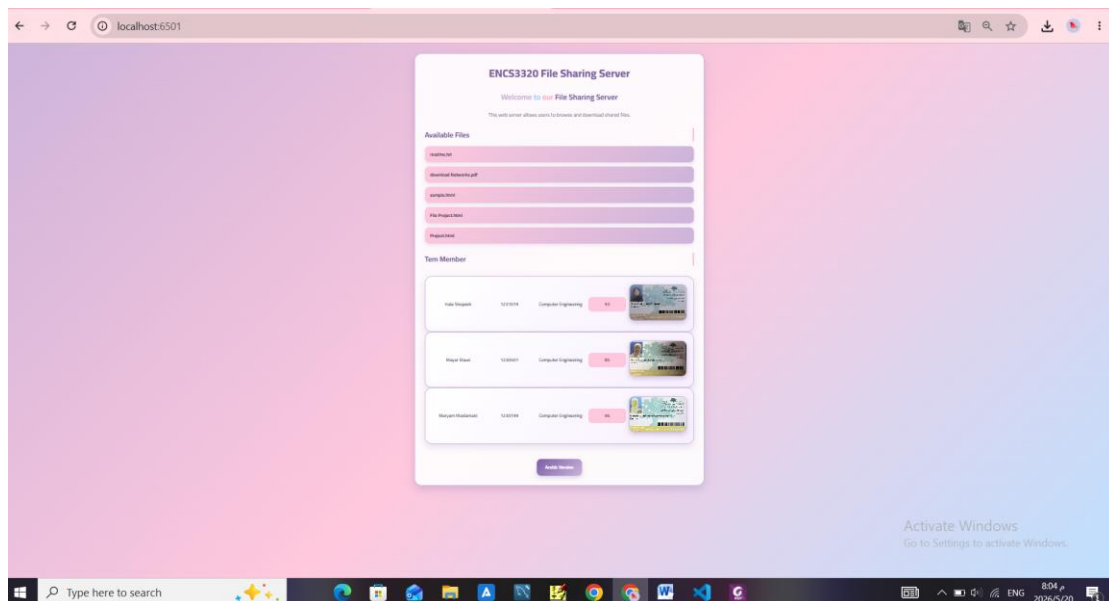


Figure 34: `http://localhost:6501/en` test


```
-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6402
Requested Path: /css/style.css
Status: 200
Total Requests: 2
Successful Requests: 2
Failed Requests: 0

REQUEST:
GET /img/Phia.jpg HTTP/1.1
Host: localhost:8081
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
sec-ch-ua: "Chromium"="140", "Google Chrome"="140", "NotABrand"="99"
sec-fetch-dest: image
sec-fetch-mode: no-cors
sec-fetch-site: same-origin
Accept: image/webp,image/svg+xml,image/*,*/*;q=0.8
Referer: http://localhost:8081/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US;q=0.9,ur;q=0.8

-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6402
Requested Path: /img/Phia.jpg
Status: 200
Total Requests: 3
Successful Requests: 3
Failed Requests: 0

REQUEST:
GET /img/Myan.jpg HTTP/1.1
Host: localhost:8081
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
sec-ch-ua: "Chromium"="140", "Google Chrome"="140", "NotABrand"="99"
sec-fetch-dest: image
sec-fetch-mode: no-cors
sec-fetch-site: same-origin
Accept: image/webp,image/svg+xml,image/*,*/*;q=0.8
Referer: http://localhost:8081/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US;q=0.9,ur;q=0.8
```

Figure 38: Terminal Output (2) – English test

```
-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6402
Requested Path: /img/Myan.jpg
Status: 200
Total Requests: 4
Successful Requests: 4
Failed Requests: 0

REQUEST:
GET /img/Myan.jpg HTTP/1.1
Host: localhost:8081
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
sec-ch-ua: "Chromium"="140", "Google Chrome"="140", "NotABrand"="99"
sec-fetch-dest: image
sec-fetch-mode: no-cors
sec-fetch-site: same-origin
Accept: image/webp,image/svg+xml,image/*,*/*;q=0.8
Referer: http://localhost:8081/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US;q=0.9,ur;q=0.8

-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6403
Requested Path: /img/Myan.jpg
Status: 200
Total Requests: 5
Successful Requests: 5
Failed Requests: 0

REQUEST:
LOGGING
```

Figure 39: Terminal Output (3) – English test

```
-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6403
Requested Path: /img/Myan.jpg
Status: 200
Total Requests: 6
Successful Requests: 6
Failed Requests: 0

REQUEST:
LOGGING
```

Figure 40: Terminal Output (4) – English test

The server processed a total of 6 HTTP requests, where 5 requests were successfully handled and returned with a 200 OK status. These successful requests included the main page and automatically loaded static files such as CSS and images.

Only one request was rejected with a 400 Bad Request error because it was invalid or not properly formatted. As a result, the server refused to process it and counted it as a failed request.

Arabic Page Tests

The following tests ensure that the English main page is correctly served for different valid URLs.

- Accessing Arabic Page:

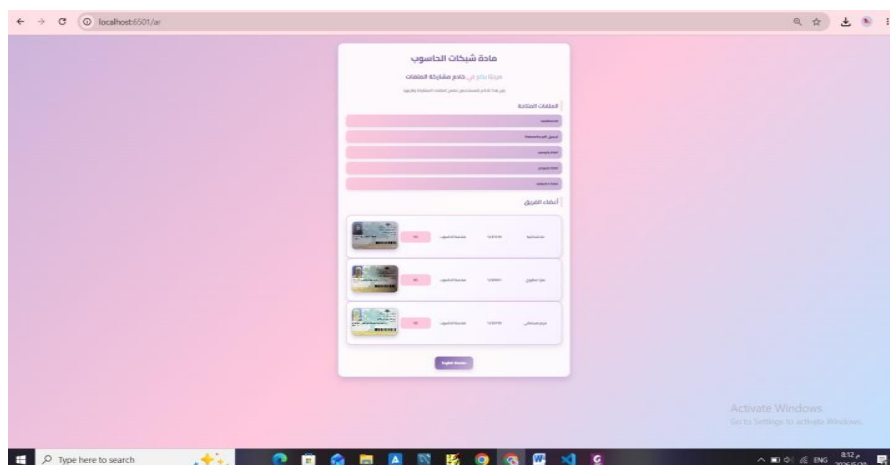


Figure 41: <http://localhost:6501/ar> test

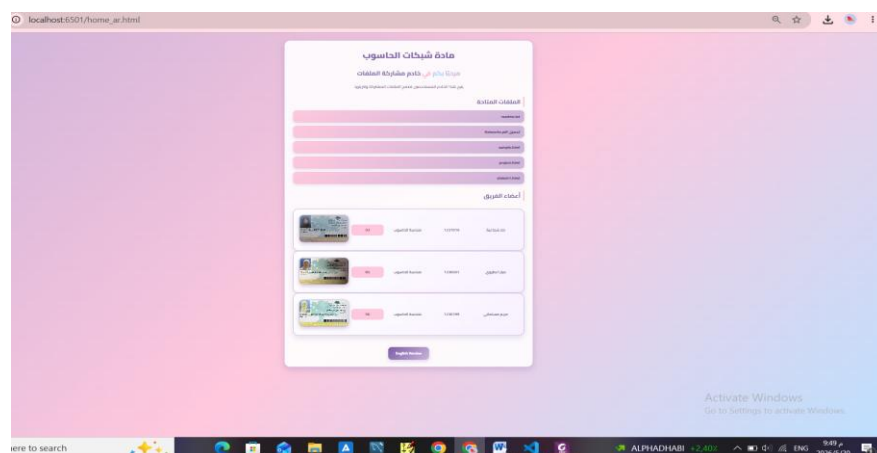


Figure 42: http://localhost:6501/home_ar.html test

Terminal output:

```
REQUEST:
GET / HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-Purpose: prefetch:previewer
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/png,*/*;q=0.8
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /
Status: 200
Total Requests: 1
Successful Requests: 1
Failed Requests: 0

REQUEST:
GET /css/style.css HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-Purpose: prefetch:previewer
Accept: text/css,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: stylesheet
Referer: http://localhost:8080/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /css/style.css
Status: 200
Total Requests: 2
Successful Requests: 2
Failed Requests: 0
```

Figure 43: Terminal output (1) – Arabic test

```
Status: 200
Total Requests: 2
Successful Requests: 2
Failed Requests: 0

REQUEST:
GET /img/Photo.jpg HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-Purpose: prefetch:previewer
Accept: image/webp,image/*;q=0.8,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8080/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /img/Photo.jpg
Status: 200
Total Requests: 3
Successful Requests: 3
Failed Requests: 0

REQUEST:
GET /img/Photo.jpg HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-Purpose: prefetch:previewer
Accept: image/webp,image/*;q=0.8,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8080/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /img/Photo.jpg
Status: 200
Total Requests: 4
Successful Requests: 4
Failed Requests: 0
```

Figure 44: Terminal Output (2) – Arabic test

```
-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /img/Photo.jpg
Status: 200
Total Requests: 4
Successful Requests: 4
Failed Requests: 0

REQUEST:
GET /img/Photo.jpg HTTP/1.1
Host: localhost:8080
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-Purpose: prefetch:previewer
Accept: image/webp,image/*;q=0.8,*/*;q=0.1
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8080/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

-----
LOGGING:
Client IP: 127.0.0.1
Client Port: 6050
Requested Path: /img/Photo.jpg
Status: 200
Total Requests: 5
Successful Requests: 5
Failed Requests: 0
```

Figure 45: Terminal Output (3) – Arabic test

The server processed a total of 5 HTTP requests, all of which were successfully handled and returned with a 200 OK status. These requests included the main /ar page and its related static resources such as CSS files and images, which were automatically loaded by the browser.

No failed requests were recorded during this run, indicating that all requests were valid and properly processed by the server.

Successful file Open (200 OK)

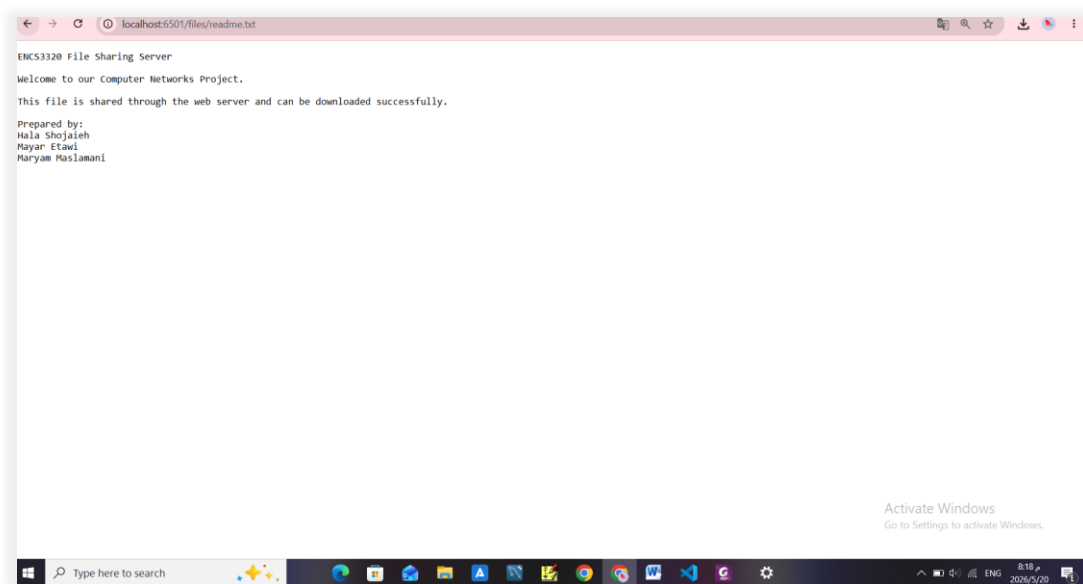


Figure 46: 200 OK – Successful File Request in Browser

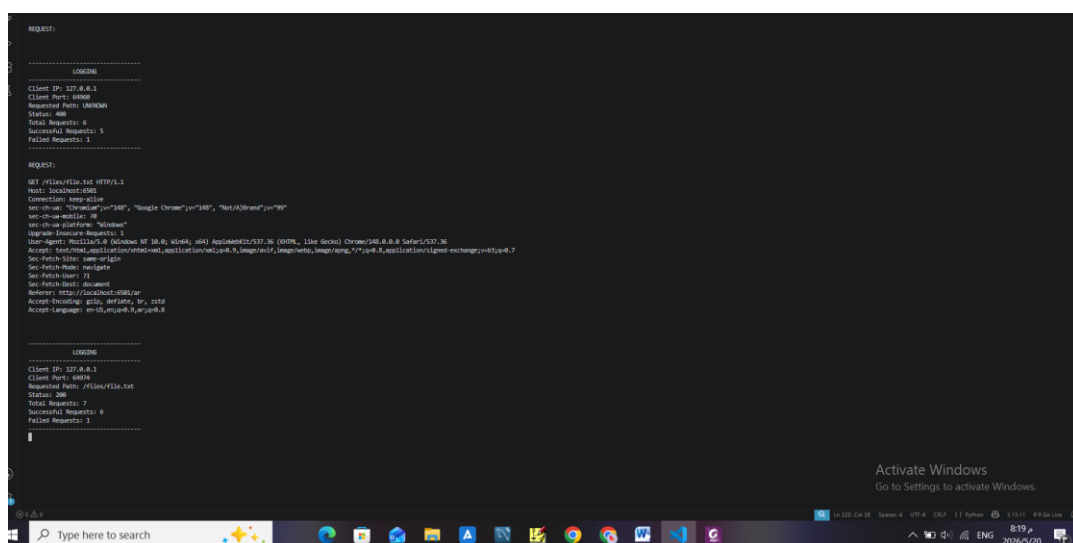


Figure 47: 200 OK – terminal output

A total of 7 HTTP requests were processed. The first 5 requests were for loading the Arabic page (/ar) and its related resources, all successfully handled with a 200 OK status. After that, the file /files/file.txt was requested and successfully served. One request was rejected with a 400 Bad Request error due to an invalid or incomplete request.

Error case (404 Not Found)

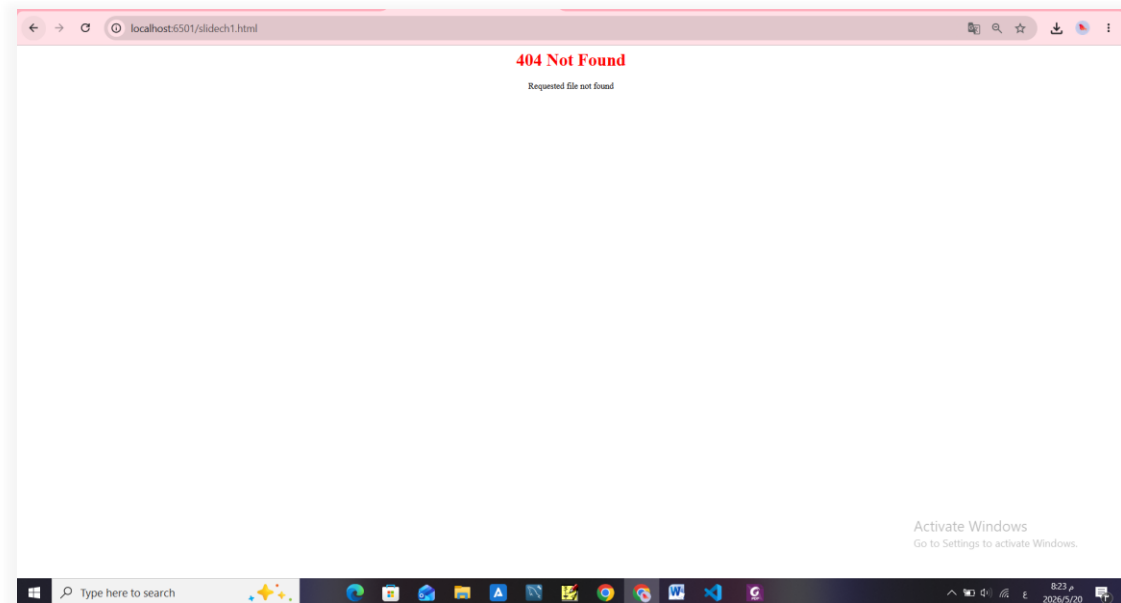


Figure 48: 404 Not Found – Browser Response for Non-Existing File Request

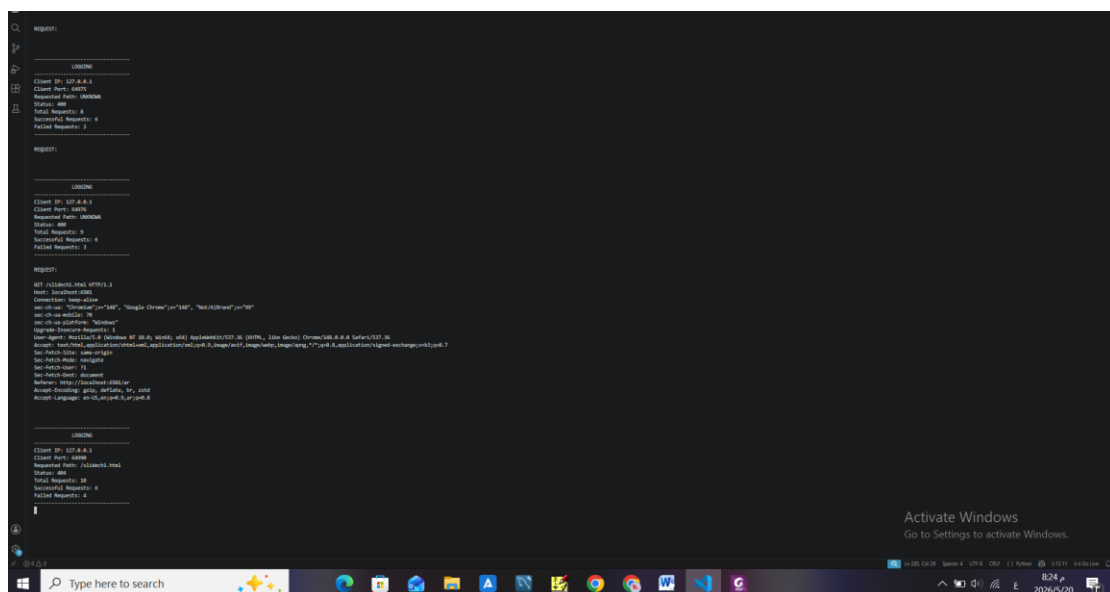


Figure 49: 404 Not Found – Terminal Output

A total of 10 HTTP requests were processed. The first 7 requests were used to load the Arabic homepage (/ar) and its related resources, all successfully handled with a 200 OK status, including files and images loaded by the browser. After that, a file request was made and served successfully when available.

Two requests were rejected with a 400 Bad Request error due to invalid or incomplete requests, and one request for /slidech1.html failed with a 404 Not Found error because the file does not exist on the server.

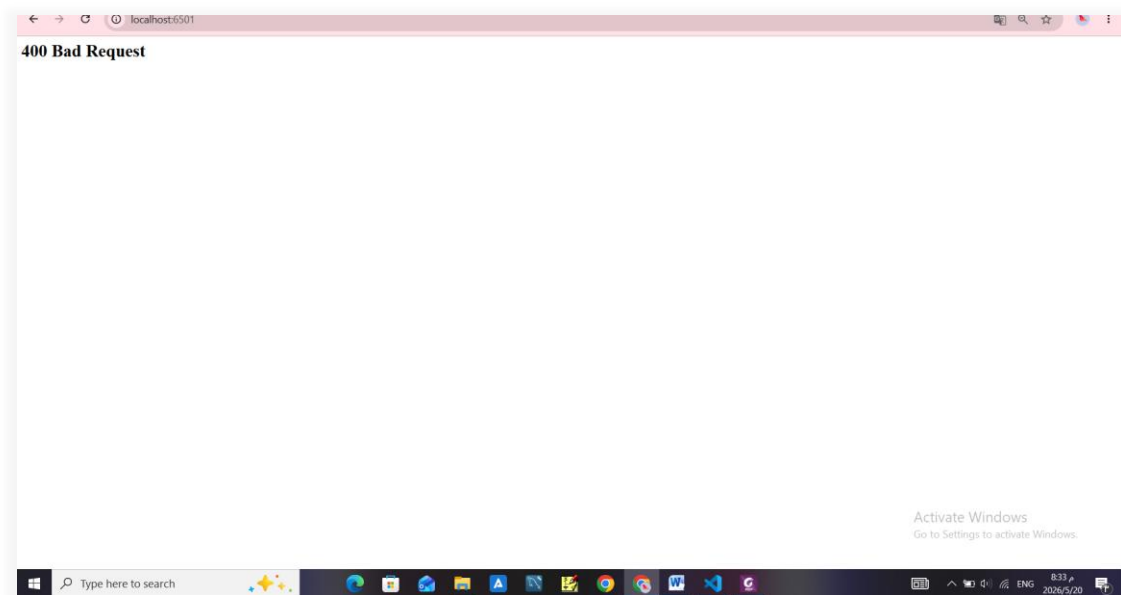


Figure 50: 400 Bad Request – Browser Response for Malformed Request

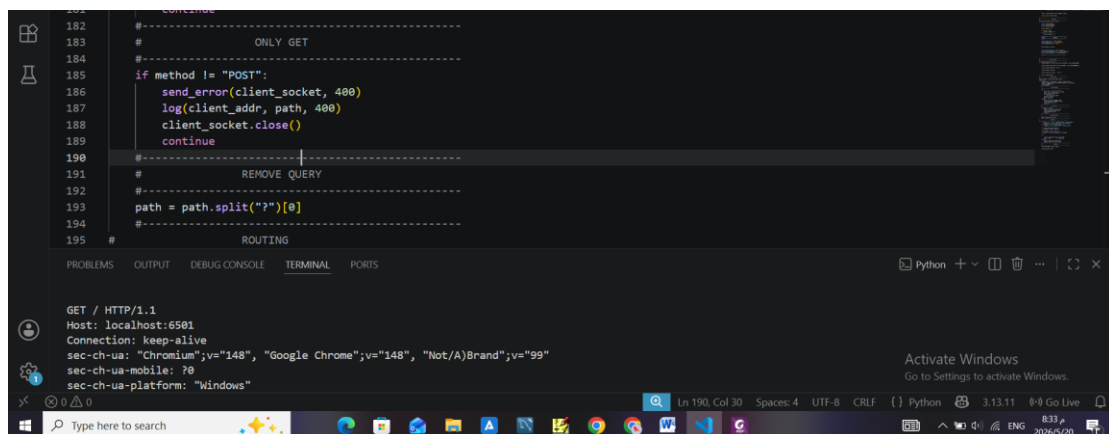


Figure 51: 404 Not Found – Terminal Output (1)

```
REQUEST:
GET / HTTP/1.1
Host: localhost:6061
Connection: keep-alive
sec-ch-ua: "Chrome";v="140", "Google Chrome";v="140", "Not(A)Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-FromSource: yes
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.5,ar;q=0.8

-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6061
Requested Path: /
Status: 400
Total Requests: 1
Successful Requests: 0
Failed Requests: 1

REQUEST:
GET / HTTP/1.1
Host: localhost:6061
Connection: keep-alive
sec-ch-ua: "Chrome";v="140", "Google Chrome";v="140", "Not(A)Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
Sec-FromSource: yes
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.5,ar;q=0.8

-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6061
Requested Path: /
Status: 400
Total Requests: 1
Successful Requests: 0
Failed Requests: 1
```

Figure 52: 404 Not Found – Terminal Output (2)

```
-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6060
Requested Path: /
Status: 400
Total Requests: 2
Successful Requests: 0
Failed Requests: 2

REQUEST:
-----
LOGGING
Client IP: 127.0.0.1
Client Port: 6060
Requested Path: /
Status: 400
Total Requests: 3
Successful Requests: 0
Failed Requests: 3
```

Figure 53: 404 Not Found – Terminal Output (3)

Naturally, it was not possible to generate a Bad Request directly from the browser because it always sends valid HTTP GET requests. Therefore, this case was tested by modifying the server function to treat any request that is not a proper GET request or is malformed as an error and respond with a 400 Bad Request. This allowed to demonstrate how the server handles invalid requests and terminates the connection appropriately.

Conclusion

This project provided practical experience in computer networking concepts through the implementation and testing of different networking tools and protocols. CMD networking commands were used to analyze network configuration, connectivity, DNS resolution, routing, and active connections. In addition, TCP and UDP client/server applications were implemented using socket programming to demonstrate the differences between reliable connection-oriented communication and connectionless communication. Finally, a simple file-sharing web server was developed using TCP sockets and HTTP request/response handling, including support for file serving, error handling, logging, and bilingual web pages. Overall, the project successfully demonstrated the practical operation of networking protocols, socket programming, and web server communication concepts.

References

- [1] J. Spencer, "PracticallyNetworked," [Online]. Available:
<https://www.practicallynetworked.com/networking-commands/> [Accessed 14 May 2026].
- [2] J. F. Kurose and Keith W. Ross, in *COMPUTER NETWORKING*, Pearson, 2020.
- [3] "MDN," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> [Accessed 20 May 2026].
- [4] "CloudFlare," [Online]. Available:
<https://www.cloudflare.com/learning/ddos/glossary/tcp-ip/> [Accessed 20 May 2026].